A Note on the Security of ECDSA with Presignatures or Biased Nonces

Geoffroy Couteau*† Nikolaos Makriyannis‡

October 19, 2026

Abstract

Over the years, several ECDSA security experiments have been proposed to enable faster distributed ECDSA signing. These experiments are not known to reduce to the standard ECDSA game, and recent work (Groth and Shoup, Eurocrypt 2022) shows that some are vulnerable, especially when combined with raw signing (signing hash digests directly, rather than pre-hashed messages) and additive key derivation. Furthermore, several of these variants are implemented in public libraries and used by companies, sometimes in commercially deployed products, yet lack sound security foundations.

In this note, we revisit the security landscape of these alternative modes of operation for ECDSA. As our main result, we show that the security of some of these variants, which are used in the wild, is equivalent in the generic group and random oracle model to a standard conjecture from fine-grained complexity on the average-case hardness of the k-SUM problem. We complement our study by proving lower bounds, conditioned on fine-grained complexity conjectures, on the running time of generic group model attacks against other standard modes of operation analyzed in (Groth and Shoup, Eurocrypt 2022). Overall, our results provide a complete picture of the security landscape of ECDSA variants, and reveal an elegant connection with (old and new) conjectures from the field of fine-grained complexity.

*CNRS, IRIF, Université Paris Cité. couteau@irif.fr

†Work done while the author was working as a consultant for Fireblocks.

‡Fireblocks. nikos@fireblocks.com

Contents

1 Introduction **3**
- 1.1 Our Contribution . . . 5
- 1.2 Related Work: Rerandomized Presignatures . . . 8
- 1.3 Organization . . . 8

2 Preliminaries **9**
- 2.1 The Elliptic-Curve GGM . . . 9
- 2.2 Presignatures . . . 9
  - 2.2.1 Mapping Biased-Nonce ECDSA to Security Game 1. . . . 11
- 2.3 Known Attacks on Presignatures . . . 11
- 2.4 The k-SUM Problem . . . 12

3 A New Attack on Rerandomized Presignatures **13**
- 3.1 An Attack on Security Game 2 . . . 14

4 Rerandomized Presignatures: 3-MMUL to Forgery Reduction **14**
- 4.1 Preliminaries . . . 15
- 4.2 Lazy Symbolic Evaluation . . . 16
- 4.3 Reduction from (q,t)-3MMUL . . . 18
- 4.4 Reduction from Hash Inverse . . . 19

5 Plain Presignatures: 4-MMUL to Forgery Reduction **21**
- 5.1 Preliminaries . . . 21
- 5.2 Plain Presignatures w/o Key-Derivation . . . 22
- 5.3 Plain Presignatures w/ Key-Derivation . . . 23

6 Fine-Grained Average-Case Hardness of kMMUL **24**
- 6.1 Preliminaries on k-SUM . . . 24
- 6.2 Reducing k-MMUL to k-SUM at Density 1 . . . 25
  - 6.2.1 Step 1. . . . 25
  - 6.2.2 Step 2. . . . 26
  - 6.2.3 Step 3. . . . 26
  - 6.2.4 Step 4. . . . 27

References **27**

A GGM Forgery to Lazy Evaluation Forgery Reduction **30**
- A.1 Lazy Evaluation 1 . . . 31
- A.2 Lazy Evaluation 2 . . . 32
- A.3 Symbolic Evaluation 1 . . . 34
- A.4 Symbolic Evaluation 2 . . . 36

B Linear-Time HashInv Solver **39**

1 Introduction

ECDSA is one of the most widely deployed signature schemes in existence, underpinning much of the security infrastructure of the Internet. It is also the dominant signature scheme in the blockchain ecosystem, securing hundreds of millions of transactions every day—the setting that will be our focus, since the ECDSA variants we study are, to our knowledge, deployed almost exclusively there. ECDSA signatures have the form (r, σ), where r is derived from the nonce—essentially a random point on an elliptic curve—and σ depends on the nonce, the secret key, and the message to be signed. Concretely, given a secret key $x \leftarrow \mathbb{Z}_q$ and a public key $X = g^x \in \mathbb{G}$ over an elliptic curve $\mathbb{G}$ of order q generated by g, an ECDSA signature (r, σ) on a message m is obtained by letting $R = g^k$ be a public nonce, and $\sigma = (\mathsf{F}(m) + x \cdot r) \cdot k^{-1} \bmod q$, where r denotes the first coordinate of the point R on the curve, and F denotes the ECDSA message digest function (implemented with the SHA family of hash functions).

Threshold ECDSA is one of the most widely deployed protocols for protecting ECDSA keys and is at the core of numerous commercial solutions [Fir25; Coi25; Dkl; Pro25; Blo] enabling various distributed-wallet applications such as treasury management, Wallet-as-a-Service (WaaS) [Fir] and embedded wallets [Dyn; ABCGJM24], among others. In threshold ECDSA, the key is shared among multiple parties who jointly run an interactive protocol to produce signatures. Despite its popularity and widespread use, ECDSA is notoriously hard to evaluate in a distributed setting. All existing protocols are resource-intensive, incurring high costs in at least one of the following: computation, communication, or round complexity.

Presignatures and Biased Nonces. To remedy this situation, several works have introduced alternative *modes of operation* for ECDSA. These modes of operation fall in two categories:

- *Presignatures*, introduced independently in [CMP20; GG20; DOKSS20; DJNPØ20], let the nonce be precomputed before the message to be signed is available, enabling extremely fast, succinct, and non-interactive signature generation once the message is given.
- *Biased nonces*, initially introduced in [DKLs18] and recently revisited in [ABCGJM24], sidestep the need for a commit-and-open phase in threshold ECDSA by letting one party see its opponent's share of the nonce before choosing its own, enabling efficient *round-optimal* protocols (one offline round, one online round).

Within each category, the modes differ by whether, and how, they integrate a *rerandomization* step to attempt to strengthen their security guarantees: early presignature proposals let the participants construct a shared nonce R (via Diffie-Hellman agreement) and set the final nonce $\hat{R}$ to $\hat{R} = R$. Others, such as [Fir25], rerandomize the nonce multiplicatively as $\hat{R} = R^{\mu}$ for a random μ (derived after constructing R via a random oracle query). More recently, Groth and Shoup [GS22] suggested using additive rerandomization instead: $\hat{R} = R \cdot g^{\mu}$. Similarly, known biased-nonce modes use different rerandomizers: Doerner et al. [DKLs18] set $\hat{R} = R \cdot R_1^{\delta}$ for random δ, where $R_1 = g^{k_1}$, $R_2 = g^{k_2}$, and $R = g^{k_1 k_2}$, with k_1 held by the first-message sender (initiator) in the protocol and k_2 by the second-message sender (responder). The recent work of Adjedj et al. [ABCGJM24] instead uses $\hat{R} = R \cdot R_2^{\delta}$. These choices yield different adversarial capabilities and leakage profiles.

Security Concerns for ECDSA Modes of Operation. Despite their efficiency, these modifications are not innocuous: for all presignature and biased-nonce variants, security is *not known*

to reduce to the standard ECDSA notion. Indeed, there is good reason for this: Groth and Shoup [GS22] showed that ECDSA with presignatures requires, at a minimum, an additional assumption on the hash function[1]. This implies that each variant requires a dedicated security analysis to be confidently relied upon.

The situation is further complicated by the fact that presignature modes of operation are also often considered in conjunction with other (orthogonal) modifications of the ECDSA security game:

- In *raw signing*, the message digest—rather than the message itself—is passed as input to the signatories. Raw signing is useful to many providers, as it enables them to offer signing capabilities for variants of ECDSA with custom hash functions. It is also useful when end users wish to hide the actual contents of their messages from the provider. It is worth noting that raw signing is typically offered in HSMs [Pkc].

- With *additive key derivation*, many subkeys $X_\gamma = X \cdot g^\gamma$ can be derived from a single master key X. This mechanism is used widely in cryptocurrency applications due to the Hierarchical Deterministic Wallet scheme standardized in Bitcoin Improvement Proposal 32 (BIP32, [Wui12]).

Using presignatures (without rerandomization) in combination with these variants introduces other attack vectors: presignatures are utterly insecure in the raw-signing mode of operation, and admit faster generic attacks when they are combined with additive key derivation [GS22].

A first systematic study of the ECDSA landscape was initiated by Groth and Shoup [GS22], showing attacks on presignatures with either raw-signing or key-derivation, and proving the security in the elliptic-curve generic group model (EC-GGM, a variant of the generic group model (GGM) introduced in [Sho97] that models more closely the setting of elliptic curve groups) of additively-rerandomized presignatures (even in conjunction with raw-signing or key-derivation). In the biased-nonce setting, the scheme of Adjedj et al. [ABCGJM24] comes with a tight security proof in the EC-GGM. However, there remain multiple gaps in our understanding of the security landscape of these ECDSA variants:

- For presignatures without rerandomization, with or without additive key-derivation, it is not clear to what extent the cryptanalysis of [GS22] in the EC-GGM translates to a real-world attack, as the attack model counts only the number of queries to the generic group, but ignores the cost of all other operations. Furthermore, it is not known whether these attacks are best-possible, or whether better attacks could exist.

- For presignatures with multiplicative rerandomization [CMP20], as used in [Fir25] (and by at least one other company [com25]), no supporting security analysis is known in the literature.

- Finally, the biased nonce mode of Doerner et al. [DKLs18] (deprecated, but implemented in several public libraries [Tau21; Coi21; Qui23b] and used by some companies [Qui23a]) claims security in the GGM, but only a sketch of the analysis is provided. As we will see later, the situation for this mode is more complex than originally anticipated by Doerner et al. [DKLs18].

[1] This assumption is also implicit in Canetti et al. [CMP20], but there it is absorbed by the random oracle model.

As threshold ECDSA and its many modes of operation are routinely used to secure monetary transactions, the lack of a clear security landscape is a concern. Our aim in this work is to provide a complete picture of the security landscape. Along the way, we reveal an elegant connection of this landscape with the field of fine-grained complexity theory.

Important update: a new 3SUM algorithm

Alman and Vassilevska Williams [AV26] report a truly subquadratic algorithm for 3SUM, discovered by an LLM and subsequently checked and developed by the authors. Assuming its correctness, this refutes our 3-SUM hardness assumptions and the associated conjectured security bounds for multiplicatively rerandomized presignatures and the DKLS18 biased-nonce variant [DKLs18]. The reductions establishing a connection in both directions between forgery and average-case computational problems remain valid. Given how recent this development is, we retain the original statements subject to this qualification and defer a fuller revision pending further scrutiny.

1.1 Our Contribution

In this work, we aim to fill the remaining gaps and provide a formal and unified security analysis of all known presignatures and biased-nonce modes of operation for ECDSA. Our study reveals a close connection between the hardness of ECDSA modes of operation in the elliptic-curve generic group model and random oracle, and standard conjectures in the field of fine-grained complexity regarding the average-case hardness of the k-SUM problem. Before stating our main result, let us introduce a few notions. The *average-case k-SUM conjecture at density 1* states the following: let L be a list of $N^{1/k}$ random integers between $-N$ and N. Then no algorithm can find k integers $a_1, \cdots, a_k \in L$ such that $a_1 + \cdots + a_k = 0$ in time much faster than $N^{\lceil k/2 \rceil / k}$. The standard form of the conjecture rules out algorithms with constant success probability. A slightly stronger form of the conjecture, due to [DKK22], rules out algorithms with slightly smaller success probability $\Omega(1/\log^2 N)$. Let $\mathsf{F} : \{0,1\}^* \to \mathbb{Z}_q$ denote the ECDSA message digest function. We prove the following:

Theorem 1.1 (informal)**.** *We model F as a random oracle and let ε denote an arbitrary fixed constant. Then in the EC-GGM, there exists an attacker running in time $T = q^{1/2-\varepsilon}$ with success probability $\Omega(1)$ against the biased-nonce mode of operation from [DKLs18], or against the presignature mode of operation with multiplicative randomizer from [Fir25], if and only if the strong average-case 3-SUM conjecture at density 1 is wrong. Furthermore, if 3-SUM at density 1 can be solved in near-optimal time $\tilde{O}(N^{1/3})$, these two modes of operation can be broken in time $T = \tilde{O}(q^{1/3})$ in the EC-GGM.*

The hardness of the average-case 3-SUM problem at density 1 is a standard and well-studied hardness assumption in the field of fine-grained complexity. A statement of the conjecture can be found for example in [DKK22, Conjecture 1.2], or in [ASSVV24, Conjecture 1.1] (the conjecture is stated there for a modular variant of k-SUM, but the variants are equivalent). Other statements of the same conjecture can be found in [Set15; LLW19; BSDV21; AHY25; DLSVW25]. The stronger form is due to [DKK22], where it was noted that even a refutation of this stronger variant would imply a major breakthrough, since the slightly sub-constant winning probability would suffice in

known cryptanalytic applications of k-SUM algorithms. This implies that finding any attack that breaks these modes of operation faster than the best known attacks on standard ECDSA would entail a major breakthrough in fine-grained complexity.

We note that a byproduct of our analysis is a proof that these two modes of operation fall short of the $q^{1/2}$ security bar in a "pure-GGM" model, where the adversary runtime is counted solely via GGM queries: both modes admit a forgery using only $q^{1/3}$ queries to the GGM (in contrast, the best known attack on plain ECDSA requires solving a discrete logarithm and takes time $q^{1/2}$). This contradicts a claim of Doerner et al. [DKLs18] that their alternative mode admits "a proof in the generic group model that the adversary has no advantage in the forgery game, relative to the standard functionality". This could be a concern, since this protocol is still maintained in some public libraries [Qui23b] and used by some companies [Qui23a], even though the authors of [DKLs18] deprecated their protocol (for reasons that include other unrelated security concerns, and because it was superseded by the protocol of Doerner et al. [DKLs24]). This is also the case for the protocol of [Fir25], which is actually deployed commercially. Fortunately, barring a breakthrough in fine-grained complexity, our analysis shows that this pure-GGM attacker cannot be turned into a real-world attack[2].

We complement our analysis by investigating the original "plain" presignature mode of operation (without rerandomizer), which was previously shown in [GS22] to be broken in the EC-GGM using $q^{1/3}$ queries, or even $q^{1/4}$ queries when the mode is combined with additive key-derivation. We observe that the attacks from [GS22] are pure-GGM attacks (that ignore all attacker costs beyond group queries), and we ask whether they can be turned into real-world attacks. We prove the following:

Theorem 1.2 (informal)**.** *There exists a $q^{1/3}$-time attack against the plain presignature mode of operation, with or without additive key-derivation, that uses the group generically. Furthermore, assuming that F is modeled as a random oracle and the strong average-case 4-SUM conjecture at density 1, there exist no $q^{1/3-\varepsilon}$-time attacks on this mode without additive key-derivation. Assuming additionally a new (modular) affine collision (AFF) conjecture, there exist no $q^{1/3-\varepsilon}$-time attacks even with additive key-derivation.*

The new AFF conjecture states, informally, that given two lists L_1, L_2 of random elements of $\mathbb{Z}_q$, no $q^{1/3-\varepsilon}$-time adversary can find two lines $(a, b) \in L_1 \times L_2$, $(c, d) \in L_1 \times L_2$, and a point $x \in L_2$, such that both lines coincide on x: $ax + b = cx + d \bmod q$ (see Security Game 4 and Conjecture 5.1 for a formal statement). One way to attack AFF is by reduction to a 4-SUM instance: pick an arbitrary $x \in L_2$, and define $L_3 = \{u \cdot x \ : \ u \in L_1\}$. Then, a solution to AFF is given by a 4-tuple $(u, v, w, y) \in L_3 \times L_2 \times L_3 \times L_2$ such that $u + v - w - y = 0 \bmod q$, which can be obtained in time $q^{1/3}$ (ignoring other polynomial factors) using Wagner's 4-list algorithm [Wag02] or some optimized variants over $\mathbb{Z}_q$ [JKL24]. Our conjecture essentially states that this is the best possible attack.

We note that the attack (and lower bound) matches that of [GS22] in the setting without key-derivation. In the setting with key-derivation, our results indicate that the plain presignature mode is plausibly more secure than the pure-GGM attack of Groth and Shoup [GS22] suggests, if a new average-case fine-grained hardness assumption holds; we leave the study of this conjecture for future work. We summarize all known results on the security of ECDSA modes of operation, including our new results, in Table 1.

[2]From a theoretical point of view, clarifying the security guarantees of [DKLs18] is also interesting since this protocol is not fully superseded by that of Doerner et al. [DKLs24]: it requires only two rounds of communication, whereas the latter requires three.

Table 1: Summary of security status of presignatures & biased nonces. "Secure" means that no attack can run in time significantly less than $\sqrt{q}$ in the generic group model, where q denotes the group size. A q^x-query attack denotes a "pure-GGM" attack, where all costs beyond GGM queries are ignored. 3-SUM and 4-SUM refer to Conjecture 2.5 instantiated with $k = 3$ and $k = 4$ respectively, and k-MUL indicates the use of an algorithm (typically Wagner's k-list) to solve the kMMUL problem (Security Game 3 and 4). AFF refers to Conjecture 5.1.

	Mode	From	Status[e]
[GS22]	Presignatures $\hat{R} = R$	[CMP20; GG20] [DOKSS20; DJNPØ20]	✗ $q^{1/3}$-query attack (3-MUL)
	Presignatures (🥩)[a]	[GS22]	✗ Insecure
	Presignatures (🗝)[a]	[GS22]	✗ $q^{1/4}$-query attack (4-SUM)
	Presignatures rerand. $\hat{R} = R \cdot g^{\mu}$	[GS22]	✓ Secure[b,c]
[ABCGJM24]	Biased nonce $R \cdot R_2^{\delta}$	[ABCGJM24]	✓ Secure[c]
	Presignatures rerand. $\hat{R} = R \cdot g^{\mu}$	[GS22]	✓ Secure[c]
[This work]	Presignatures $\hat{R} = R$	[CMP20; GG20] [DOKSS20; DJNPØ20]	⊖ $q^{1/3}$-secure[d] under 4-SUM conj.
	Presignatures (🗝)[a]	[GS22]	⊖ $q^{1/3}$-secure[d] under {4-SUM + AFF} conj.
	Biased nonce $R \cdot R_1^{\delta}$	[DKLs18]	✗ $q^{1/3}$-query attack (3-MUL)
	Presignatures rerand. $\hat{R} = R^{\mu}$	[Fir25]	✓ Secure[c] under 3-SUM conj.

[a] 🥩 and 🗝 denote the raw-signing and key-derivation modes respectively.
[b] Security holds under the assumption that rerandomization occurs once or the presignature is discarded, i.e., the attacker cannot query multiple values of μ before settling on a message.
[c] Security holds for 🥩- and 🗝-modes (including in combination). For 🗝, there is $1/d$-loss of security where d denotes the number of key-derivation paths.
[d] No security in 🥩-mode.
[e] Every attack listed uses a *single* presignature and a single signing query; the stated q^x costs are offline (hash evaluations and group operations). The exception is our attack on multiplicatively rerandomized presignatures (Section 3), which additionally makes $\Theta(q^{1/3})$ nonce-finalization queries against that one presignature, each of which requires interaction with the honest signers.

Interpreting Our Attacks. Three caveats are in order. First, every attack in this work assumes an adversary that chooses the message to be signed *adaptively*, after seeing the presignature and, where applicable, the modifier. Whether this is achievable depends on the deployment: an adversary rarely has full control over the signed messages, though we consider it realistic that it can often

strongly influence them. Second, the key-derivation attacks require the number d of available derivation paths to be large; deployments may bound d, but bounds large enough to matter are plausible in practice, e.g. under BIP32 [Wui12]. Third, a mode admitting a $q^{1/3}$-time attack is not broken in any practical sense—it offers less security than standard ECDSA, but the resulting security level may still be adequate. We reserve the word "insecure" for modes that admit a forgery with a constant number of queries.

1.2 Related Work: Rerandomized Presignatures

As highlighted in Table 1 and detailed in Section 2.3, plain (non-rerandomized) presignatures incur a meaningful security loss relative to standard ECDSA, and they are utterly insecure in raw-signing mode.

In an effort to recover the security level of ECDSA while retaining raw signing, Groth and Shoup [GS22] introduce rerandomized presignatures: the presignature R is rerandomized to a *modified* point $\hat{R}$, and the released signature uses nonce $\hat{R}$ instead of R. Groth and Shoup [GS22] propose *additive* rerandomization, setting $\hat{R} = R \cdot g^{\delta}$ for a public random δ. They show that if a single modifier δ is sampled and bound to one message digest, so only one modifier can be used and the presignature is discarded otherwise, then additive rerandomized presignatures match the security of standard ECDSA. However, this mitigation must be implemented with care: in Groth and Shoup [GS22], once the adversary chooses a message and presignature, the signature is generated and the presignature is consumed. This setting is only realistic when a public random beacon is available. In contrast, using the standard practice of performing rerandomization via a hash function (hashing the transcript *à la* Fiat-Shamir) would be insecure, since it allows the adversary to "backtrack" and reject a rerandomization outcome that is not to their liking. Unfortunately, in many scenarios, a public trusted random beacon is too strong a setup assumption.

As a consequence, several threshold ECDSA libraries [Fir25] implement a variant of rerandomized presignatures that differs from what is prescribed in Groth and Shoup [GS22], and the limited security guarantees in that work do not extend to these variants. Additionally, while Groth and Shoup [GS22] prescribe that the nonce be randomized using an *additive* public shift, multiple implementations instead apply a *multiplicative* factor.

In the recent work [ABCGJM24] on biased nonces (biased nonces and presignatures are essentially equivalent as we explain in Section 2.2), the authors show that even when an attacker can sample many different *additive* modifiers for different message digests (and even multiple modifiers for the same digest), with each modifier the output of a hash modeled as a random oracle, there is essentially no security loss relative to standard ECDSA. However, the analysis of Adjedj et al. [ABCGJM24] only applies to *additive* modifiers, and, as far as we know, no such analysis exists for multiplicative modifiers or biased nonces à la [DKLs18].

1.3 Organization

In Section 2, we introduce preliminaries on the generic group model, presignatures (including an equivalence to biased nonces and a discussion and overview of known attacks), and the k-SUM problem. Section 3 presents a new attack on rerandomized presignatures. Section 4 introduces our main reduction from the security of our general presignature experiment to a multiplicative variant of the k-SUM problem, while Section 5 extends the reduction to the setting of plain presignatures (without rerandomization), under stronger fine-grained complexity conjectures. Finally, Section 6

shows that the conjectured hardness of the multiplicative variants of k-SUM used in our work is implied by the standard "average-case k-SUM at density 1" conjecture.

2 Preliminaries

Notation. We assume some familiarity with ECDSA and basic group theory and we use multiplicative notation for group operations. We use the notation from [ABCGJM24]. Let $(\mathbb{G}, g, q)$ and $\mathsf{F} : \{0,1\}^* \to \mathbb{Z}_q$ denote the group-generator-order tuple and the message digest function associated with ECDSA. We recall that $\mathbb{G}$ is equipped with a 'conversion function' $\mathcal{X} : \mathbb{G} \to \mathbb{Z}_q$, the projection to the first coordinate, that deterministically maps group elements to elements modulo q with the property that $\mathcal{X}(A) = \mathcal{X}(A^{-1})$ for $A \in \mathbb{G}$. ECDSA public keys are simply elements $X = g^x \in \mathbb{G}$ where $x \xleftarrow{\$} \mathbb{Z}_q^*$ is drawn uniformly at random from the set of units modulo q. ECDSA signatures for a message digest $m = \mathsf{F}(\text{msg})$ consist of pairs $(r, s) \in \mathbb{F}_q^2$ such that $r = \mathcal{X}(g^k)$ and $s = k^{-1}(m + rx) \bmod q$ for $k \xleftarrow{\$} \mathbb{Z}_q^*$ chosen uniformly at random.

2.1 The Elliptic-Curve GGM

For defining the GGM, we follow [ABCGJM24], which in turn uses the EC-generic group model of [GS22] that captures some idiosyncrasies of elliptic curves, in particular the fact that $\mathcal{X}(A) = \mathcal{X}(A^{-1})$ for every $A \in \mathbb{G}$. A generic group oracle O is defined via a random bijective map $\pi : \mathbb{G} \to \boldsymbol{G}$ under the following notation and constraints.

1. $\boldsymbol{G}$ denotes the set of identifying labels of elements in group $\mathbb{G}$.
2. There are efficient algorithms for sampling uniformly from $\boldsymbol{G}$ and efficiently deciding membership in $\boldsymbol{G}$.
3. $\pi(AB) = \mathsf{O}(\pi(A), \pi(B))$ and $\mathcal{X}(\pi(A)) = \mathcal{X}(\pi(A^{-1}))$, for every A, $B \in \mathbb{G}$.

2.2 Presignatures

In this section, we define presignatures and their security game. We present all variants within a single framework. To that end, the specification in Security Game 1 uses intentionally inefficient operations—specifically how the leakage function f and the modifier ν are computed—purely to support a unified presentation. We then show how Security Game 1 can be implemented efficiently, depending on the desired mode of operation.

Additionally, for clarity—and since the main case of interest is rerandomized presignatures in raw-signing (🥩) mode—we color-code in teal the lines that apply only to the non-raw and key-derivation (🗝️) modes. These can be ignored on a first pass.

Security Game Presignatures Game

1. *Parameters.* ECDSA digest func. F and leakage func. $f : \mathbb{G}^2 \to \mathbb{Z}_q$.
2. *Key-Generation.* SO samples $x \xleftarrow{\$} \mathbb{Z}_q$ and returns $X = g^x$.
   %In 🗝️ mode, a random set of public tweaks $\boldsymbol{\Delta} \subseteq \mathbb{Z}_q$ is also returned.

3. *Signing.* When prompted for signing:

   (a) On input (`prompt.presig`) from A:

   – Return $R \xleftarrow{\$} \mathbb{G}$ to A. Record (presig, R).
   %We say that R is a presignature.

   (b) On input $(\texttt{prompt.nonce}, R, m, \mathrm{id})$:

   – If there exists a record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \nu)$, return $(\hat{R}, \nu)$.
   %In the non-🥩 mode, $m = \mathsf{F}(\mathrm{msg})$ for some $\mathrm{msg} \in \{0,1\}^*$.

   – Otherwise, if there exists a record (presig, R), sample $\hat{R} \xleftarrow{\$} \mathbb{G}$, set $\nu \leftarrow f(R, \hat{R})$, record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \nu)$, and return $(\hat{R}, \nu)$.
   %In the plain-presignatures mode, $\hat{R} = R$ and $\nu = \perp$.

   – Otherwise, ignore the message.

   %In 🗝 mode, a key-tweak $\delta \in \boldsymbol{\Delta}$ is part of the input and is recorded.

   (c) On input $(\texttt{prompt.sig}, R, m, \mathrm{id})$:

   – If there exists a record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \nu)$, return σ such that $(\mathcal{X}(\hat{R}), \sigma)$ is a valid signature for m. Erase all records of (presig, R) and $(\mathsf{nonce}, R, m, \ldots)$.
   %In 🗝 mode, σ is valid for the tweaked public key $X \cdot g^{\delta}$.

   – Otherwise, ignore the message.

4. *Outcome.* A wins if it outputs $(\mathrm{msg}_0, r_0, \sigma_0)$ such that (r_0, σ_0) is a valid signature for msg_0 and $(\texttt{prompt.sig}, \ldots, \mathsf{F}(\mathrm{msg}_0) = m_0)$ was never queried to SO. The pair (r_0, σ_0) is referred to as a forgery.

   %In 🗝 mode, A wins if the forgery is valid for any tweaked key.

Security Game 1: General Unforgeability Experiment

Definition 2.1 (Efficient Additively Rerandomized Presignatures)**.** Additively rerandomized presignatures correspond to the variant of Security Game 1 where $\nu = f(\hat{R}, R)$ and $\hat{R} = R \cdot g^{\nu}$. For an efficient implementation, the challenger samples $k \xleftarrow{\$} \mathbb{Z}_q$ and sets $R = g^k$ on a presignature query. On a nonce query, it samples $\nu \xleftarrow{\$} \mathbb{F}_q$ and sets $\hat{R} = R \cdot g^{\nu}$. It records k and ν to compute the corresponding signature when requested: $s = (\nu{+}k)^{-1} \cdot \big(m{+}\mathcal{X}(\hat{R}){\cdot}x\big)$ (or $s = (\nu{+}k)^{-1} \cdot \big(m{+}\mathcal{X}(\hat{R}){\cdot}(x{+}\delta)\big)$ in key-derivation mode).

Definition 2.2 (Efficient Multiplicatively Rerandomized Presignatures)**.** Multiplicatively rerandomized presignatures correspond to the variant of Security Game 1 where $\nu = f(\hat{R}, R)$ and $\hat{R} = R^{\nu}$. For an efficient implementation, the challenger samples $k \xleftarrow{\$} \mathbb{Z}_q$ and sets $R = g^k$ on a presignature query. On a nonce query, it samples $\nu \xleftarrow{\$} \mathbb{F}_q$ and sets $\hat{R} = R^{\nu}$. It records k and ν to compute the corresponding signature when requested: $s = (\nu k)^{-1} \cdot \big(m + \mathcal{X}(\hat{R}) \cdot x\big)$ (or $s = (\nu k)^{-1} \cdot \big(m + \mathcal{X}(\hat{R}) \cdot (x + \delta)\big)$ in key-derivation mode).

2.2.1 Mapping Biased-Nonce ECDSA to Security Game 1.

While not immediate, Security Game 1 also captures the biased-nonce setting in threshold ECDSA. In this regime, the initiator sends $R_1 = g^{k_1}$; the responder samples $R_2 = g^{k_2}$ and finalizes the nonce as a deterministic function of its view (the function includes a cryptographic hash for security). Below we show that, modeling the hash as a random oracle, the security of biased nonces is captured by Security Game 1. We stress that the biased-nonce effect does not affect an honest responder (the nonce is unbiased from their perspective), so the analysis considers only a corrupted responder with an honest initiator.

DKLs18 Modifier. Doerner et al. [DKLs18] set the nonce to $\hat{R} = R_1^{k_2+\delta}$, where δ is the random-oracle output on (R_2, R', m, X) and $R' = g^{k_1 k_2}$. Viewing $R = R_1$ as a presignature, since k_2 can be extracted via a knowledge-of-exponent (KEA) assumption,[3] we can program the oracle so that $\nu = k_2 + \delta$. Then $\hat{R} = R_1^{k_2+\delta} = R^{\nu}$, which matches a multiplicatively rerandomized presignature.

ABC+24 Modifier. Adjedj et al. [ABCGJM24] set the nonce to $\hat{R} = (R_1 \cdot g^{\delta})^{k_2}$, where δ is the random-oracle output on (R_2, R, m, X) and $R = g^{k_1 k_2}$. Viewing R as a presignature, since k_2 can be extracted via a knowledge-of-exponent (KEA) assumption, we can program the oracle so that $\nu = \delta \cdot k_2$. Then $\hat{R} = (R_1 \cdot g^{\delta})^{k_2} = R \cdot g^{k_2 \delta} = R \cdot g^{\nu}$, which matches an additively rerandomized presignature.

2.3 Known Attacks on Presignatures

In this section, we describe known attacks for the various presignature modes. In the following, $R \in \mathbb{G}$ denotes a presignature requested by the attacker at the beginning of Security Game 1. We write $r = \mathcal{X}(R)$.

Plain Presignatures and Raw Signing. The attacker A aims to calculate a forgery $\mathsf{F}(\mathrm{msg}_0) = m_0 \in \mathbb{Z}_q$. For this purpose, A samples a random element $R_0 = R^{\lambda} \in \mathbb{G}$ and sets $r_0 = \mathcal{X}(R_0)$. Then, it requests a signature for $m = m_0 \cdot r/r_0$, and obtains s for the signature. It sets $s_0 = s r_0 \cdot (r\lambda)^{-1}$. It follows that the pair (r_0, s_0) is a forgery for m_0.[4] Indeed:

$$R_0^{s_0} = (R^s)^{r_0/r} = (g^m X^r)^{r_0/r} = g^{m_0} X^{r_0}.$$

Plain Presignatures w/o Key-Derivation. When disallowing the raw-signing capability, the above attack is thwarted, as it is no longer possible to request signatures for digests that are not outputs of the ECDSA hash F (indeed, the attack boils down to finding $\mathrm{msg} \in \{0,1\}^*$ such that $\mathsf{F}(\mathrm{msg}) = m_0 \cdot r/r_0$). Still, as pointed out by Groth and Shoup [GS22], there remains an attack vector not present in plain ECDSA: it suffices to find msg and msg_0 such that $\mathsf{F}(\mathrm{msg})/\mathsf{F}(\mathrm{msg}_0) = r/r_0$. Furthermore, by iterating over many possible group elements R_0, it suffices for the adversary to find

[3] If the attacker returns a DH tuple (R_1, R_2, R') for a random R_1, then k_2 is extractable under KEA, and unconditionally in the GGM.

[4] This attack may seem meaningless: the attacker already has access to a raw signing oracle that returns signatures on chosen messages. However, it has practical ramifications in the derived-keys setting, where the attacker may request a signature under a related key $X \cdot g^{\delta}$ with public δ. In this case, a signature on $m = m_0 - \delta r$ under $X \cdot g^{\delta}$ is also a valid signature on m_0 under X.

msg, msg_0 and λ such that $\mathsf{F}(\text{msg})/\mathsf{F}(\text{msg}_0) = \mathcal{X}(R)/\mathcal{X}(R^\lambda)$. From a pure query-complexity point of view, solutions are guaranteed to exist with constant probability with as few as $q^{1/3}$ queries.

Plain Presignatures w/ Key-Derivations. Key derivation opens up an alternative attack opportunity that only uses a single presignature R. By querying many values $m_i = \mathsf{F}(\text{msg}_i)$ and key tweaks $\delta_j \in \boldsymbol{\Delta}$, the attacker is tasked with finding m_{i_1}, m_{i_2} and $\delta_{j_1}, \delta_{j_2}$ such that $m_{i_1} + \delta_{j_1} r = m_{i_2} + \delta_{j_2} r$ for $r = \mathcal{X}(R)$. If successful, the attacker requests a signature for m_{i_1} on tweak δ_{j_1} which immediately yields a valid signature for m_{i_2} on tweak δ_{j_2}. Indeed, letting s denote the value returned by the signing oracle:

$$R^s = g^{m_{i_1}} \cdot (X \cdot g^{\delta_{j_1}})^r = g^{m_{i_1} + r\delta_{j_1}} X^r = g^{m_{i_2} + r\delta_{j_2}} X^r = g^{m_{i_2}} \cdot (X \cdot g^{\delta_{j_2}})^r.$$

Groth and Shoup [GS22] show how to solve this problem using Wagner's algorithm in query *and* time complexity $q^{1/3}$, while a pure-GGM attack is possible with as few as $q^{1/4}$ queries.

2.4 The k-SUM Problem

Definition 2.3 (Average-case k-SUM problem)**.** An input to the (N, r)-kSUM problem is given by a list L of r integers sampled uniformly and independently over $\{-N, \cdots, N\}$. The goal is to find a k-tuple (an ordered set of distinct indices) $(i_1, \cdots, i_k)$ such that $L[i_1] + \cdots + L[i_k] = 0$, where the sum is over $\mathbb{Z}$.

Instances of the average-case k-SUM problem are usually characterized by their *density* regime. The density of a k-SUM instance (N, r)-kSUM is defined as $\Delta := \log \binom{r}{k} / \log N \approx k \log r / \log N$ (where the approximation is very close to reality for low values of k; in this work, we will always consider $k \in \{3, 4\}$). At density $\Delta = 1$, a kSUM instance has exactly one solution on average. In the dense regime $\Delta > 1$, many solutions exist, while in the sparse regime $\Delta < 1$, solutions are unlikely to exist. A folklore and well-studied conjecture states that k-SUM is average-case hard at density 1:

Conjecture 2.4 (Average-case k-SUM conjecture)**.** *Any algorithm that solves the* $(N, N^{1/k})$-kSUM *problem with probability* $\Omega(1)$ *must run in expected time* $T = \Omega(N^{\lceil k/2 \rceil / k - o(1)})$.

A statement of the above conjecture can be found for example in [DKK22, Conjecture 1.2], or in [ASSVV24, Conjecture 1.1] (the conjecture is stated there for a modular variant of kSUM, but as we will see later, the variants are equivalent). Other statements of the same conjecture (or generalizations thereof) can be found in [Set15; LLW19; BSDV21; AHY25; DLSVW25]. In our work, we will rely on a slight strengthening of this conjecture:

Conjecture 2.5 (Average-case k-SUM conjecture at $\varepsilon = 1/\log^2 N$)**.** *Any algorithm that solves the* $(N, N^{1/k})$-kSUM *problem with probability* $\Omega(1/\log^2 N)$ *must run in expected time* $T = \Omega(N^{\lceil k/2 \rceil / k - o(1)})$.

Important update: consequences for the conjectures

Assuming the correctness of [AV26], Conjectures 2.4 and 2.5 are false for $k = 3$: on $N^{1/3}$ inputs, the reported algorithm finds a zero-sum triple in $\tilde{O}(N^{1.9992/3})$ time, with constant success probability over our input distribution. Together with the algorithmic connections discussed in Section 6, this also contradicts the $k = 3$ case of Conjecture 6.4 and the corresponding $q^{1/2-o(1)}$ security claims. These consequences do not refute the $k = 4$ or AFF conjectures.

This strengthening has been explicitly put forth in [DKK22]. Conjecture 2.5 is also implied by the following conjecture (very close in spirit to the "traditional" k-SUM conjecture at density 1): no algorithm can $(N^{\lceil k/2 \rceil/k-o(1)}, \Omega(1))$-solve the $(N, \log^2 N \cdot N^{1/k})$-kSUM problem, which is an instance of k-SUM at density $\Delta = 1 + o(1)$.

3 A New Attack on Rerandomized Presignatures

In this section, we present a new attack on the multiplicatively rerandomized presignature experiment. For simplicity and for the benefit of the reader, we restate Security Game 1 but this time focusing on raw and non-key-derived modes.

Security Game Mult. Rerandomized Presignatures Game

1. *Parameters.* ECDSA digest func. F.
2. *Key-Generation.* SO samples $x \xleftarrow{\$} \mathbb{Z}_q$ and returns $X = g^x$.
3. *Signing.* When prompted for signing:
   (a) On input (`prompt.presig`) from A:
      – Sample $k \leftarrow \mathbb{Z}_q^*$ and return $R = g^k$ to A. Record (presig, R, k).
   (b) On input $(\texttt{prompt.nonce}, R, m, \mathrm{id})$:
      – If there exists a record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \mu)$, return $(\hat{R}, \mu)$.
      – Otherwise, if there exists a record (presig, R), sample $\mu \xleftarrow{\$} \mathbb{Z}_q$, set $\hat{R} = R^\mu$, record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \mu)$, and return $(\hat{R}, \mu)$.
      – Otherwise, ignore the message.
   (c) On input $(\texttt{prompt.sig}, R, m, \mathrm{id})$:
      – If there exists a record $(\mathsf{nonce}, R, m, \mathrm{id}, \hat{R}, \mu)$, return σ such that $\sigma = (k\mu)^{-1}(m + \mathcal{X}(\hat{R}) \cdot x)$. Erase all records of (presig, R, k) and $(\mathsf{nonce}, R, m, \ldots)$.
      – Otherwise, ignore the message.
4. *Outcome.* A wins if it outputs $(\mathrm{msg}_0, r_0, \sigma_0)$ such that (r_0, σ_0) is a valid signature for msg_0 and $(\texttt{prompt.sig}, \ldots, \mathsf{F}(\mathrm{msg}_0) = m_0)$ was never queried to SO. The pair (r_0, σ_0) is referred to as a forgery.

Security Game 2: Unforgeability Experiment

3.1 An Attack on Security Game 2

Definition 3.1. Let $t \in \mathbb{N}$ and define A in Security Game 2 by the following specification.

1. Set $m \leftarrow 1$.
2. Make a single (`prompt.presig`) query: obtain R.
3. Make $t/3$ distinct queries $\{(\texttt{prompt.nonce}, R, m, \mathrm{id}_i)\}_{i \in [t/3]}$.

   Write μ_i for each modifier and $r_i = \mathcal{X}(R^{\mu_i})$.
4. Compute R^γ for $\gamma = 1, \ldots, t/3$. Let $f_\gamma = \mathcal{X}(R^\gamma)$.
5. Compute $e_j = \mathsf{F}(\mathrm{msg}_j)$ for $t/3$ arbitrary distinct messages msg_j.
6. Find $\gamma, i, j \in [t/3]$ such that $e_j \cdot r_i = f_\gamma$.
7. Query $(\texttt{prompt.sig}, R, m, \mathrm{id}_i)$. Obtain s.
8. Output $((f_\gamma, \sigma), \mathrm{msg}_j)$ for $\sigma = s\mu_i e_j / \gamma$.

Claim 3.2. *It holds that $((f_\gamma, \sigma), \mathrm{msg}_j)$ is a forgery, i.e., $F_\gamma^\sigma = g^{\mathsf{F}(\mathrm{msg}_j)} X^{f_\gamma}$ for $F_\gamma = R^\gamma$. Furthermore, in the GGM and random oracle models, the query complexity of A is exactly t.*

Proof. $F_\gamma^\sigma = R^{\gamma\sigma} = R^{s\mu_i e_j} = ((R^{\mu_i})^s)^{e_j} = (g^m X^{r_i})^{e_j} = g^{\mathsf{F}(\mathrm{msg}_j)} X^{f_\gamma}$ □

Remark 3.3. To produce a forgery, A must find a suitable triple (e_j, r_i, f_γ). We ignored this search cost above. If obtaining such a triple were free, the attacker would forge with constant probability for $t = q^{1/3}$, since suitable tuples exist for this t. However, as we show next, this is essentially the only attack, and the cost of finding such a triple is prohibitively high (assuming a long-standing conjecture).

4 Rerandomized Presignatures: 3-MMUL to Forgery Reduction

In this section, we present the main reduction from 3MMUL to forgery in the GGM, where 3MMUL denotes the problem below. In Section 6, we reduce the conjectured hardness of 3MMUL (Conjecture 6.4) to a standard conjecture in fine-grained complexity (Conjecture 2.4).

Security Game (q, t)-3MMUL problem

Parameters. A 2ℓ-bit prime q, a list size t.

Challenge. The challenger sends $L \leftarrow \mathbb{Z}_q^t$ to $\mathcal{A}$.

Output. $\mathcal{A}$ outputs a triple $(i_1, i_2, i_3) \in [t]^3$. The challenger outputs 1 if $L[i_1] \cdot L[i_2] \cdot L[i_3] = 1 \bmod q$, and 0 otherwise.

Security Game 3: The (q, t)-3MMUL problem

Part of our analysis follows the GGM analysis of [ABCGJM24]. After introducing some preliminaries from [ABCGJM24], we define three experiments for the reduction. The first, the lazy symbolic evaluation experiment, is an intermediate step that emulates the GGM while tracking all group elements as formal random variables. Up to a negligible loss $< 5t^2/q$ (where t is the adversary's running time), this experiment is indistinguishable from the GGM; see Appendix A for the proof. The second experiment, Algorithm 2, is our main reduction from 3MMUL to Forgery. The third, Algorithm 3, reduces *Hash-Inversion to Forgery*: given any forger for the GGM, we build an algorithm that inverts the hash. This handles edge cases not covered by Algorithm 2.
The symbolic descriptions below admit a direct implementation with $O(tu)$ bookkeeping, where $u \leq t$ bounds the number of yet-to-be-signed presignatures. As shown in Appendix B, the reduction can be implemented so that a forgery attacker running in time t yields an attacker for the underlying problem running in time $O(t)$, measured in field operations and expected dictionary operations, with an additional $O(t^2/q)$ error. The appendix treats the HashInv solver; the 3MMUL solver and the variants in Section 5 can be handled analogously. We use these linear-time implementations for the fine-grained runtime claims. Only for hash inversion in the ROM does the query bound alone suffice, irrespective of the bookkeeping cost.

4.1 Preliminaries

Definition 4.1. For $\mathbb{D} = \{(\alpha, \beta, \gamma, \zeta, m, r, e) \in \mathbb{Z}_q^7 \text{ s.t. } \alpha\zeta \neq e\beta\}$, define $\mathsf{ssig} : \mathbb{D} \to \mathbb{Z}_q$ such that

$$\mathsf{ssig} : (\alpha, \beta, \gamma, \zeta, m, r, e) \mapsto \gamma \cdot \frac{er - \zeta m}{\alpha\zeta - e\beta}$$

Claim 4.2. *Fix $\alpha, \beta, \gamma, \zeta, m, r \in \mathbb{Z}_q$ such that $\gamma, \zeta \neq 0$. Define the function $\mathsf{ssig}' : e \mapsto \mathsf{ssig}(\alpha, \beta, \gamma, \zeta, m, r, e) \in \mathbb{Z}_q$ where e ranges over $\mathbb{D}' = \{e \in \mathbb{Z}_q \text{ s.t. } (\alpha, \beta, \gamma, \zeta, m, r, e) \in \mathbb{D}\}$. If $r\alpha - \beta m \neq 0$, then ssig' is* injective.

Proof.

$$\begin{aligned}
\mathsf{ssig}'(e_1) = \mathsf{ssig}'(e_2) &\iff \\
\frac{e_1 r - \zeta m}{\alpha\zeta - e_1\beta} = \frac{e_2 r - \zeta m}{\alpha\zeta - e_2\beta} &\iff \\
e_1 r\alpha\zeta - e_1 r e_2\beta - \zeta^2 m\alpha + \zeta m e_2\beta = e_2 r\alpha\zeta - e_2 r e_1\beta - \zeta^2 m\alpha + \zeta m e_1\beta &\iff \\
e_1 r\alpha\zeta + \zeta m e_2\beta = e_2 r\alpha\zeta + \zeta m e_1\beta &\iff \\
\zeta(e_1 - e_2)(r\alpha - \beta m) = 0 &\iff \\
e_1 = e_2 &
\end{aligned}$$

□

We will be using the following lemma from [GS22].

Lemma 4.3. *For the identifying labels $\boldsymbol{G}$ of an elliptic curve encoded by the tuple $(\mathbb{G}, g, q)$, there exists an efficient randomized algorithm* Samp *such that for $A \xleftarrow{\$} \mathsf{Samp}(x)$ where x is drawn uniformly from $\mathbb{Z}_q^*$:*

1. $\Pr[A \in \{\mathsf{fail}\} \cup \boldsymbol{G}] = 1$ *and* $\Pr[A = \mathsf{fail}] \leq \frac{3}{4} + \frac{q^{-1/2}}{\sqrt{2}}$
2. *If $A \in \boldsymbol{G}$, then $\mathcal{X}(A) = x$.*
3. *For any $P \in \boldsymbol{G} \setminus \{\mathbb{1}\}$, $\Pr[A = P \mid A \neq \mathsf{fail}] = 1/(q-1)$.*

Parameter Choice (Size of q). Hereafter, we assume that q is sufficiently large such that $1 - \left(\frac{3}{4} + \frac{q^{-1/2}}{\sqrt{2}}\right) \geq 1/5$ and $(1 - 1/q) \geq 1/5$. A straightforward calculation shows that this condition is satisfied for any $q \geq 2^8$.

4.2 Lazy Symbolic Evaluation

We begin with a lazy evaluation that tracks all group elements as formal variables, rather than assigning discrete logs on demand. The experiment SymbLazyEval maintains two mappings, π for group elements and λ for modifiers, which are consulted whenever the attacker queries a group element or a modifier. It also keeps two lists: $\boldsymbol{R}$ for presignatures and $\boldsymbol{V}$ for the formal variables representing their discrete logs. When a signature is requested from SymbLazyEval, the experiment returns a random string s and updates π so that future group-oracle queries remain consistent with its bookkeeping.
Additionally, SymbLazyEval handles queries to the ECDSA message-digest function F, modeled as a random oracle. It returns random values consistent with a mapping η it maintains locally.

Algorithm SymbLazyEval

1. Initialization.
   (a) $\pi \leftarrow \{(0, \mathbb{1})\}$
   (b) Sample $g \xleftarrow{\$} \boldsymbol{G}$ and set $\pi \leftarrow \pi \cup \{(1, g), (-1, g^{-1})\}$.
   (c) Sample $X \xleftarrow{\$} \boldsymbol{G}$ and $\pi \leftarrow \pi \cup \{(\mathfrak{R}_1, X), (-\mathfrak{R}_1, X^{-1})\}$. (If $X \in \mathrm{range}(\pi)$, abort)
   (d) Set $\boldsymbol{V} \leftarrow \{\mathfrak{R}_1\}$ and $\boldsymbol{R} \leftarrow \varnothing$ (the presignatures set) and $c = 1$.
   (e) For all $P \in \boldsymbol{G}$ such that $\mathcal{X}(P) = 0$, execute the addition query on input $(P, \mathbb{1})$.
   (f) Return (g, X)
2. Hash query. On input $\mathrm{msg} \in \{0,1\}^*$, do:
   (a) If $\mathrm{msg} \notin \mathrm{domain}(\eta)$
      i. Sample $h \xleftarrow{\$} \mathbb{Z}_q$.
      ii. Set $\eta \leftarrow \eta \cup \{(\mathrm{msg}, h)\}$.
   (b) Return $\eta(\mathrm{msg})$
3. Group query. On input $i = i_0 + \sum_{\mathfrak{R} \in \boldsymbol{V}} i_{\mathfrak{R}} \cdot \mathfrak{R}$, do:

(a) If $i \notin \text{domain}(\pi)$

i. Sample $P \xleftarrow{\$} \boldsymbol{G}$. (If $P \in \text{range}(\pi)$, abort)

ii. Set $\pi \leftarrow \pi \cup \{(i, P), (-i, P^{-1})\}$.

(b) Return $\pi(i)$

4. Group addition. On input $(P_1, P_2) \in \boldsymbol{G}^2$, do:

(a) For $j \in \{1, 2\}$, if $P_j \notin \text{range}(\pi)$:

i. Sample $i \xleftarrow{\$} \mathbb{Z}_q^*$. (If $i \in \text{domain}(\pi)$, abort)

ii. Set $\pi \leftarrow \pi \cup \{(i, P_j), (-i, P_j^{-1})\}$.

(b) Return the group query output on input $\pi^{-1}(P_1) + \pi^{-1}(P_2)$.

5. Presignatures. Set $c \leftarrow c + 1$, and do:

(a) Sample $R \xleftarrow{\$} \boldsymbol{G}$. (If $R \in \text{range}(\pi)$, abort).

(b) Set $\pi \leftarrow \pi \cup \{(\mathfrak{R}_c, R), (-\mathfrak{R}_c, R^{-1})\}$, $\boldsymbol{V} \leftarrow \boldsymbol{V} \cup \{\mathfrak{R}_c\}$ and $\boldsymbol{R} \leftarrow \boldsymbol{R} \cup \{R\}$.

(c) Return R.

6. Modifier. On input $(R, m, \text{id}) \in \boldsymbol{R} \times \mathbb{Z}_q^* \times \{0,1\}^*$, do:

(a) If $(R, m, \text{id}) \notin \text{domain}(\lambda)$

i. Sample $\mu \xleftarrow{\$} \mathbb{Z}_q^*$. (If $\mu \cdot \pi^{-1}(R) \in \text{domain}(\pi)$, abort)

ii. Invoke the group query on input $\mu \cdot \pi^{-1}(R)$.

iii. Set $\lambda \leftarrow \lambda \cup ((R, m, \text{id}), \mu)$.

(b) Return $\lambda(R, m, \text{id})$

7. Signature. Letting $\mathfrak{R} \in \boldsymbol{V}$ be such that $\pi(\mathfrak{R}) = R$, on input $(R, m, \text{id}) \in \text{domain}(\lambda)$, do:

(a) Let $\mu = \lambda(R, m, \text{id})$. Obtain $\hat{R}$ by invoking the group query on input $\mu \cdot \mathfrak{R}$.

(b) Set $\hat{r} = \mathcal{X}(\hat{R})$ and sample $s \xleftarrow{\$} \mathbb{Z}_q^*$.

(c) Substitute $s^{-1}(m + \hat{r} \cdot \mathfrak{R}_1)$ for $\pi^{-1}(\hat{R})$ throughout $\text{domain}(\pi)$.

Abort if any two polynomials collapse, i.e., there's a collision.

(d) Remove all records of R and $\mathfrak{R}$ from $\boldsymbol{R}$, $\boldsymbol{V}$ and λ.

(e) Return $(\hat{r}, s)$

Algorithm 1: Symbolic Lazy Evaluation

Theorem 4.4. *Assume that* A *breaks the unforgeability of Security Game 2 in the GGM and ROM with probability ε, then,* A *breaks the unforgeability in the symbolic lazy evaluation experiment with probability $\varepsilon - \text{err}$ where $\text{err} \approx 5t^2/q$ and the exact error term is specified in Theorem A.1.*

Proof. See proof of Theorem A.1. □

4.3 Reduction from (q,t)-3MMUL

In this section, we describe the reduction to 3MMUL. Upon receiving a list L of t random elements in $\mathbb{Z}_q$ from the (q,t)-3MMUL challenger, we describe a simulator that emulates the symbolic evaluation from Section 4.2 with the purpose of extracting a 3MMUL witness from the forgery. The simulator will carefully inject the values from the list into its interaction with the attacker. The differences from SymbLazyEval are highlighted in red.

Algorithm 3MMUL Solver

Input. $t \in \mathbb{N}$ and a 3MMUL instance $L \leftarrow \mathbb{Z}_q^t$.

1. Init. Set $\mathsf{ctr} \leftarrow 0$.
2. Hash query, $\mathrm{msg} \in \{0,1\}^*$, do:
   (a) If $\mathrm{msg} \notin \mathrm{domain}(\eta)$
      i. Increment ctr and set $h \leftarrow L[\mathsf{ctr}]$.
      ii. Set $\eta \leftarrow \eta \cup \{(\mathrm{msg}, h)\}$.
   (b) Return $\eta(\mathrm{msg})$
3. Group query. On input $i = i_0 + \sum_{\mathfrak{R} \in \boldsymbol{V}} i_{\mathfrak{R}} \cdot \mathfrak{R}$, do:
   (a) If $i \notin \mathrm{domain}(\pi)$
      i. Increment ctr and choose $Z \in \boldsymbol{G}$ such that $\mathcal{X}(Z) = L[\mathsf{ctr}]^{-1}$ using algorithm Samp from Lemma 4.3 on input $L[\mathsf{ctr}]$. If Z is not well defined or $Z \in \mathrm{range}(\pi)$, choose $Z \xleftarrow{\$} \boldsymbol{G}$ (If $Z \in \mathrm{range}(\pi)$, abort).
      ii. Set $\pi \leftarrow \pi \cup \{(i, Z), (-i, Z^{-1})\}$.
   (b) Return $\pi(i)$
4. Modifier. On input $(R, m, \mathrm{id}) \in \boldsymbol{R} \times \mathbb{Z}_q^* \times \{0,1\}^*$, do:
   (a) If $(R, m, \mathrm{id}) \notin \mathrm{domain}(\lambda)$
      i. Sample $\mu \xleftarrow{\$} \mathbb{Z}_q^*$. (If $\mu \cdot \pi^{-1}(R) \in \mathrm{domain}(\pi)$, abort)
      ii. Increment ctr and choose $Z \in \boldsymbol{G}$ such that $\mathcal{X}(Z) = L[\mathsf{ctr}] \cdot m$ using algorithm Samp from Lemma 4.3 on input $L[\mathsf{ctr}] \cdot m$. If Z is not well defined or $Z \in \mathrm{range}(\pi)$, choose $Z \xleftarrow{\$} \boldsymbol{G}$ (If $Z \in \mathrm{range}(\pi)$, abort).
      iii. Set $\pi \leftarrow \pi \cup \{(i, Z), (-i, Z^{-1})\}$ and $\lambda \leftarrow \lambda \cup ((R, m, \mathrm{id}), \mu)$.
   (b) Return $\lambda(R, m, \mathrm{id})$

Algorithm 2: 3MMUL to Forgery Reduction

Theorem 4.5. *Let* A *denote a* t*-time attacker that returns a forgery* (f, σ) *for a message digest* $m_0 = \eta(\mathrm{msg}_0)$ *for some* $\mathrm{msg}_0 \in \{0,1\}^*$ *in its interaction with Algorithm 2 and let* F *denote the unique group element satisfying* $\mathcal{X}(F) = f$ *and* $F = g^{m_0/\sigma} X^{f/\sigma}$ *(i.e.,* $\pi(m_0/\sigma + f/\sigma \cdot \mathfrak{R}_1) =$

*F). Additionally, let $F = g^{\alpha}X^{\beta}\hat{R}^{\gamma}$ denote the formal expression of F when all but one of the presignatures in F's expression were consumed, and let $\hat{m}$ denote the message digest used to consume the modified presignature $\hat{R}$ and $\hat{r} = \mathcal{X}(\hat{R})$. If $\gamma \neq 0$ and $\alpha\hat{r} - \beta\hat{m} = 0$, then, with probability $(1/5)^2 - 2t/q$, it holds that $(m_0, f^{-1}, \hat{r}/\hat{m})$ is a witness for the (q,t)-*3MMUL *instance L.*

Proof. Assuming that Samp did not return $\perp$ in the corresponding calls, it holds that there exist distinct $i_1, i_2, i_3 \in [t]$ such that $m_0 = L[i_1]$, $f = L[i_2]^{-1}$ and $\hat{r}/\hat{m} = L[i_3]$. Additionally, since $F = g^{m_0/\sigma}X^{f/\sigma}$ and $F = g^{\alpha}X^{\beta}\hat{R}^{\gamma} = g^{\alpha+\gamma(\hat{m}/s)}X^{\beta+\gamma(\hat{r}/s)}$ (where s denotes the signature associated with $\hat{R}$ and $\hat{m}$), it follows that

$$
\begin{aligned}
&\begin{cases} m_0/\sigma = \alpha + \gamma\hat{m}/s \\ f/\sigma = \beta + \gamma\hat{r}/s \end{cases} && \iff \\
&\begin{cases} (m_0/\sigma - \alpha)/\hat{m} = \gamma/s \\ (f/\sigma - \beta)/\hat{r} = \gamma/s \end{cases} && \implies \\
&\hat{r}(m_0/\sigma - \alpha) = \hat{m}(f/\sigma - \beta) && \iff \\
&\hat{r} \cdot m_0 = \hat{m}f && \iff \\
&(\hat{r}/\hat{m}) \cdot m_0 \cdot f^{-1} = 1 && \iff \\
&L[i_3] \cdot L[i_1] \cdot L[i_2] = 1.
\end{aligned}
$$

The third-to-last equality uses the fact that $\alpha\hat{r} - \beta\hat{m} = 0$. To conclude the proof, we note that Samp fails with probability at most $1/5$ and we pay an additional $2t/q$ term for the probability that L contains the zero element or that $Z \in \text{range}(\pi)$ in Algorithm 2. □

4.4 Reduction from Hash Inverse

Algorithm HashInv Solver

Input. $t \in \mathbb{N}$ and a HashInv instance $e \xleftarrow{\$} \mathbb{Z}_q$.

1. Initialization. Sample $j_0 \xleftarrow{\$} [t]$ and toss a coin $b \leftarrow \{0,1\}$.
2. Hash query. On input msg $\in \{0,1\}^*$, return F(msg).
3. Group query. On input $i = \alpha + \beta \cdot \mathfrak{R}_1 + \sum_{\mathfrak{R} \in \boldsymbol{V} \setminus \{\mathfrak{R}_1\}} i_{\mathfrak{R}} \cdot \mathfrak{R}$, do:
   (a) If $i \notin \text{domain}(\pi)$
      i. If the attacker makes its j_0-th query:
         A. **If** $b = 0$: Choose $Z \in \boldsymbol{G}$ such that $\mathcal{X}(Z) = \alpha^{-1}e\beta$ using algorithm Samp from Lemma 4.3 on input $\alpha^{-1}e\beta$. If Z is not well defined or $Z \in \text{range}(\pi)$, go to Item 3(a)ii.
         B. **If** $b = 1$: Let $\boldsymbol{S} \leftarrow \{\mathfrak{R} \text{ s.t. } i_{\mathfrak{R}} \neq 0\} \setminus \{\mathfrak{R}_1\}$, sample $Z \xleftarrow{\$} \boldsymbol{G}$ and set $Z_0 \leftarrow Z$ (If $Z \in \text{range}(\pi)$, abort).
      ii. Else, choose $Z \xleftarrow{\$} \boldsymbol{G}$ (If $Z \in \text{range}(\pi)$, abort) .
   Set $\pi \leftarrow \pi \cup \{(i, Z), (-i, Z^{-1})\}$

(b) Return $\pi(i)$

4. Signature. Letting $\mathfrak{R}$ be such that $\pi(\mathfrak{R}) = R$, on input $(R, \delta, z, m) \in \text{domain}(\lambda)$, do:

   (a) Let $\mu = \lambda(R, \delta, z, m)$. Obtain $\hat{R}$ by invoking the group query on input $\mu + z \cdot \mathfrak{R}$.

   (b) Set $\hat{r} = \mathcal{X}(\hat{R})$ and $\hat{m} = m + \hat{r}(\delta - \delta_0)$.

   (c) i. **If** $b = 1$: If $\boldsymbol{S} = \{\mathfrak{R}\}$, $Z_0 = g^{\alpha} X^{\beta} \hat{R}^{\gamma}$, $\hat{r}\alpha - \beta\hat{m} \neq 0$, set $s = \mathsf{ssig}(\alpha, \beta, \gamma, \mathcal{X}(Z), \hat{m}, \hat{r}, e)$. If these conditions are not fulfilled or $s = 0$, set $s \xleftarrow{\$} \mathbb{Z}_q^*$.

   ii. Else, sample $s \xleftarrow{\$} \mathbb{Z}_q^*$.

   (d) Substitute $s^{-1}(\hat{m} + \hat{r} \cdot \mathfrak{R}_1)$ for $\mu + z\mathfrak{R}$ throughout $\text{domain}(\pi)$.

   Abort if any two polynomials collapse, i.e., there's a collision.

   (e) Remove all records of R and $\mathfrak{R}$ from $\boldsymbol{R}$, $\boldsymbol{V}$, $\boldsymbol{S}$ and λ.

   (f) Return $(\hat{r}, s)$

Algorithm 3: Hash Inverse to Forgery Reduction

Theorem 4.6. *Let* A *denote a* t*-time attacker that returns a forgery* (f, σ) *for a message digest* $m_0 = \eta(\text{msg}_0)$ *for some* $\text{msg}_0 \in \{0,1\}^*$ *in its interaction with Algorithm 3, and define* $F, \hat{R}, \hat{m}, \hat{r}, \alpha, \beta, \gamma$ *as in Theorem 4.5. If* $\gamma = 0$ *or* $\alpha\hat{r} - \beta\hat{m} \neq 0$*, then, with probability* $1/(10t) - t/q$*, it holds that* msg_0 *is a witness for the* $\mathsf{HashInv}$ *instance* e*.*

Proof. We begin with the case $\gamma = 0$. Then either F was queried in the form $g^{\alpha}X^{\beta}$, or $F = R' = g^{m'/s'}X^{r'/s'}$ for some modified presignature $\hat{R}$ and signature (r', s') on m'. In the latter case, we get $\sigma = s'$ and $m_0 = m'$, which is ruled out by hypothesis (the forgery digest cannot match a previously signed digest). So next we assume that F was queried to the group oracle as $F = g^{\alpha}X^{\beta}$. Notice that $F = g^{\alpha}X^{\beta} = g^{m_0/\sigma}X^{f/\sigma}$ implies $\alpha = m_0/\sigma$ and $\beta = f/\sigma$. So, if $f = \alpha^{-1}e\beta$, it follows that $\mathsf{F}(\text{msg}_0) = \eta(\text{msg}_0) = m_0 = e$, which solves the $\mathsf{HashInv}$ instance. We conclude this case by noting that $f = \alpha^{-1}e\beta$ occurs with probability $1/(10t) - t/q$ ($b = 0$, j_0 was guessed correctly, Samp returns $Z \neq \perp$ and $Z \notin \text{range}(\pi)$).
For the case $\gamma \neq 0$, we observe that

$$\begin{cases} F = g^{\alpha} \cdot X^{\beta} \cdot \hat{R}^{\gamma} = g^{m_0/\sigma} \cdot X^{f/\sigma} \\ \hat{R} = g^{\hat{m}/s} X^{\hat{r}/s} \end{cases} \Rightarrow \begin{cases} \alpha + \hat{m}\gamma/s = m_0/\sigma \\ \beta + \hat{r}\gamma/s = f/\sigma \end{cases} \Rightarrow$$

$$f \cdot (\alpha + \hat{m}\gamma/s) = m_0 \cdot (\beta + \hat{r}\gamma/s) \Rightarrow \tag{1}$$

$$s = \gamma \cdot \frac{m_0\hat{r} - \hat{m}f}{\alpha f - m_0\beta} \quad \Rightarrow \quad s = \mathsf{ssig}(\alpha, \beta, \gamma, f, \hat{m}, \hat{r}, m_0) \tag{2}$$

Note that Equation (1) and $\alpha f - m_0\beta = 0$ implies $m_0\hat{r} - \hat{m}f = 0$ implies $\alpha\hat{r} - \hat{m}\beta = 0$, which we have ruled out by assumption. Thus, s is well defined in Equation (2). Assuming $s = \mathsf{ssig}(\alpha, \beta, \gamma, f, \hat{m}, \hat{r}, e)$, since $s = \mathsf{ssig}(\alpha, \beta, \gamma, f, \hat{m}, \hat{r}, m_0)$ and $\mathsf{ssig}' : e \mapsto \mathsf{ssig}(\alpha, \beta, \gamma, f, \hat{m}, \hat{r}, e)$ is injective in this regime of parameters, it follows that $\mathsf{F}(\text{msg}_0) = \eta(\text{msg}_0) = m_0 = e$. This concludes the proof by noting that $s = \mathsf{ssig}(\alpha, \beta, \gamma, f, \hat{m}, \hat{r}, e)$ occurs with probability $1/(10t) - t/q$ ($b = 1$, j_0 was guessed correctly, and $s \neq 0$).

□

5 Plain Presignatures: 4-MMUL to Forgery Reduction

In this section, we analyze plain, non-rerandomized presignatures. We adapt the security experiments from Section 4 and state only the differences—both in experiments and proofs—from the rerandomized-presignature case.

5.1 Preliminaries

Our analysis relies on the following two problems:

Security Game (q,t)-4MMUL problem

Parameters. A 2ℓ-bit prime q, a list size t.

Challenge. The challenger sends $L \leftarrow \mathbb{Z}_q^t$ to $\mathcal{A}$.

Output. $\mathcal{A}$ outputs a tuple $(i_1, i_2, i_3, i_4) \in [t]^4$. The challenger outputs 1 if $L[i_1] \cdot L[i_2] \cdot L[i_3] \cdot L[i_4] = 1 \bmod q$, and 0 otherwise.

Security Game 4: The (q,t)-4MMUL problem

Security Game (q,d,t)-AFF problem

Parameters. A 2ℓ-bit prime q, list sizes d and t.

Challenge. The challenger sends $(L_1, L_2) \leftarrow \mathbb{Z}_q^d \times \mathbb{Z}_q^t$ to $\mathcal{A}$.

Output. $\mathcal{A}$ outputs a tuple $(a, b, c, d', x) \in ([d] \times [t])^2 \times [t]$. The challenger outputs 1 if $L_1[a] \cdot L_2[x] + L_2[b] = L_1[c] \cdot L_2[x] + L_2[d'] \bmod q$, and 0 otherwise.

Security Game 5: The (q,d,t)-AFF problem

In Section 6, we reduce the conjectured hardness of 4MMUL to Conjecture 2.5 for $k = 4$, a standard conjecture in fine-grained complexity. For AFF, it is not too hard to see that an attacker against 4MMUL can be turned into an attacker against AFF with the same runtime (by picking a single arbitrary element $L_2[x]$); we conjecture that one cannot do better, and that AFF has hardness comparable to 4MMUL:

Conjecture 5.1. *Any algorithm $\mathcal{A}$ that solves the (q,t,t)-AFF problem with constant probability (over the random choice of L_1, L_2 and its own random coins) has expected running time at least $q^{1/3-o(1)}$.*

On the AFF Conjecture. The AFF problem is precisely the question of whether the key-derivation attack of Groth and Shoup [GS22] can be realized in time matching its query complexity. That attack seeks $m_{i_1} + \delta_{j_1} r = m_{i_2} + \delta_{j_2} r$ for a *fixed* r; solutions already exist after $q^{1/4}$ queries, yet the only known way to find one runs in time $q^{1/3}$, via Wagner's algorithm [Wag02]. Conjecture 5.1 generalizes this by letting $x = \mathcal{X}(R)$ range over many presignatures, and asserts that this additional

freedom—the very source of the $q^{1/4}$ query bound—still yields no attack running in time below $q^{1/3}$.

As for the literature, fixing any $x \in L_2$ and scaling L_1 by x reduces AFF to a 4-MSUM instance (with repeated lists and a sign pattern, both of which Wagner's algorithm handles), solvable in time $q^{1/3}$. We are not aware of this exact problem having been studied; the closest we know of are the algebraic average-case problems of Alman et al. [AHY25] and the k-SUM variants of Agrawal et al. [ASSVV24]. We leave reducing AFF to a standard fine-grained conjecture, or refuting it, as an intriguing open problem. We stress that Conjecture 5.1 is needed only for the key-derivation case; plain presignatures without key derivation rest on Conjecture 2.5 for $k = 4$ alone.

5.2 Plain Presignatures w/o Key-Derivation

Definition 5.2. Define SymbLazyEval′ to be the analogue of SymbLazyEval from Algorithm 1 with the following differences:

1. There is no modifier query.
2. Signature queries are returned for pairs (R, m) such that $m = \eta(\text{msg})$ for some previous hash query $\text{msg} \in \{0,1\}^*$ and $\hat{R} = R$ (in particular, raw message digests are not accepted for signing).

Definition 5.3 (4MMUL Solver)**.** Define 4MMUL solver with respect to SymbLazyEval′ to be the analogue of Algorithm 2 with respect to SymbLazyEval with the following differences:

1. For every hash query, the algorithm increments ctr, flips a coin, and sets h to either $L[\mathsf{ctr}]$ or $L[\mathsf{ctr}]^{-1}$ depending on the value of the coin.
2. For every presignature query, the algorithm increments ctr and returns R such that $\mathcal{X}(R) = L[\mathsf{ctr}]$ (or a random value if R is not well defined).

Theorem 5.4. *Let* A *denote a t-time attacker that returns a forgery (f, σ) for a message digest $m_0 = \eta(\text{msg}_0)$ for some $\text{msg}_0 \in \{0,1\}^*$ in its interaction with* 4MMUL *Solver and let F denote the unique group element satisfying $\mathcal{X}(F) = f$ and $F = g^{m_0/\sigma} X^{f/\sigma}$ (i.e., $\pi(m_0/\sigma + f/\sigma \cdot \mathfrak{R}_1) = F$). Additionally, let $F = g^{\alpha} X^{\beta} R^{\gamma}$ denote the formal expression of F when all but one of the presignatures in F's expression were consumed, and let m denote the message digest used to consume the presignature R and $r = \mathcal{X}(R)$. If $\gamma \neq 0$ and $\alpha r - \beta m = 0$, then, with probability $1/100 - 2t/q$, it holds that (m_0, m^{-1}, f^{-1}, r) is a witness for the (q,t)-*4MMUL *instance L.*

Proof. Suppose that $(m_0, m^{-1}, f^{-1}, r) = (L[i_1], L[i_2], L[i_3], L[i_4])$. The proof follows exactly the same lines as the proof of Theorem 4.5. In the end, we obtain

$$\begin{aligned} r \cdot m_0 = mf \quad &\iff \\ r \cdot m^{-1} \cdot m_0 \cdot f^{-1} = 1 \quad &\iff \\ L[i_4] \cdot L[i_2] \cdot L[i_1] \cdot L[i_3] = 1. & \end{aligned}$$

Noting that Samp fails with probability at most $1/5$, and that we pay a multiplicative factor $1/4$ for guessing whether m and m_0 should be inverted, the success probability is $1/100 - 2t/q$, taking into account the probability that L contains the zero element or that $Z \in \text{range}(\pi)$ in Algorithm 2. □

Dealing with Attackers That Evade the 4MMUL Solver. As in the rerandomized-presignatures case, the reduction above only applies when $\gamma \neq 0$ and $\alpha r - \beta m = 0$. To handle forgeries that violate this condition, we use a verbatim version of Algorithm 3 to reduce HashInv to Forgery. The analysis is exactly the same and the algorithm succeeds with probability $1/(10t) - t/q$. We omit the details to avoid redundancy.

5.3 Plain Presignatures w/ Key-Derivation

Definition 5.5. Define SymbLazyEval″ to be the analogue of SymbLazyEval from Algorithm 1 with the following differences:

1. There is a set $\mathbf{\Delta} \subseteq \mathbb{Z}_q$ of additive key tweaks of size $d \in \mathbb{N}$ (this set is chosen at random at the beginning of the experiment and is returned together with the public key X).
2. When sampling $X \xleftarrow{\$} \mathbb{G}$, do: sample $(X_0, \delta_0) \xleftarrow{\$} \mathbb{G} \times \mathbf{\Delta}$ and set $\pi \leftarrow \{(\mathfrak{R}_1, X_0), (-\mathfrak{R}_1, X_0^{-1}), (-\delta_0 + \mathfrak{R}_1, X), (\delta_0 - \mathfrak{R}_1, X^{-1})\}$ when updating π.
3. There is no modifier query.
4. Signature queries are answered only for triples (R, m, δ) satisfying:
   (a) $m = \eta(\text{msg})$ for some prior hash query $\text{msg} \in \{0,1\}^*$ and $\hat{R} = R$; in particular, raw digests are not accepted for signing.
   (b) $\delta \in \mathbf{\Delta}$, and the returned signature s is valid for the public key $X \cdot g^{\delta}$ (not X). Concretely, $s^{-1}(m + r \cdot (\mathfrak{R}_1 + \delta - \delta_0))$, rather than $s^{-1}(m + r \cdot (\mathfrak{R}_1 - \delta_0))$, is substituted for $\pi^{-1}(R)$ throughout domain(π).

Definition 5.6 (AFF Solver)**.** The AFF Solver takes as input two lists L_1 and L_2 of size d and t, respectively, and maintains a counter ctr that is incremented every time it is called. Define AFF by modifying SymbLazyEval″ as follows:

1. Set $\mathbf{\Delta} \leftarrow L_1$ and initialize a counter $\mathsf{ctr} = 0$.
2. For every hash query, increment ctr and return $L_2[\mathsf{ctr}]$.
3. For every presignature query, increment ctr and return R such that $\mathcal{X}(R) = L_2[\mathsf{ctr}]$ (or a random value if R is not well defined).

Theorem 5.7. *Let* A *denote a t-time attacker that returns a forgery (f, σ) for a message digest $m_0 = \eta(\text{msg}_0)$ for some $\text{msg}_0 \in \{0,1\}^*$ in its interaction with* AFF *Solver and let F denote the unique group element satisfying $\mathcal{X}(F) = f$ and $F = g^{(m_0 + f\delta_1)/\sigma} X^{f/\sigma}$. Additionally, assume that F is a presignature and hence $F = g^{(m + f\delta_2)/s} X^{f/s}$ for some $m = \eta(\text{msg})$. Then, with probability at least $1/5$, the tuple $(\delta_1, \delta_2, m_0, m, f)$ is a witness for the (q, d, t)-*AFF *instance L_1, L_2.*

Proof. The proof is straightforward from the AFF Solver's specification; the only loss in the reduction occurs when Samp fails. □

Dealing with Attackers That Evade the AFF Solver. All other attackers are handled as in Section 5.2, except the reduction must guess the key tweak on which the forgery is produced. This is modeled by $\mathsf{SymbLazyEval}''$ and related experiments: they pick a random X_0 from the tweaked-key space and set the discrete log to the formal variable $\mathfrak{R}_1$. If the guess is correct, we essentially reduce to the untweaked case. Consequently, the reductions lose a factor $1/d$ in advantage, where d is the size of the tweaked-key space.

6 Fine-Grained Average-Case Hardness of kMMUL

In this section, we show that the conjectured hardness of kMMUL for $k \in \{3, 4\}$ is implied by Conjecture 2.5, a standard conjecture in fine-grained complexity.

6.1 Preliminaries on k-SUM

Several variants of the k-SUM problem have been considered in the literature. A common variant is the *modular* k-SUM problem (that we denote k-MSUM), where the goal is to find k entries summing to 0 modulo N:

Definition 6.1 (Average-case k-MSUM problem)**.** An input to the (N, r)-kMSUM problem is given by a list L of r integers sampled uniformly and independently over $\mathbb{Z}_N$. The goal is to find a k-tuple (an ordered set of distinct indices) $(i_1, \cdots, i_k)$ such that $L[i_1] + \cdots + L[i_k] = 0 \bmod N$.

We will rely on two powerful lemmas from [DKK22] that relate the (average-case) hardness of k-SUM at density 1 to its hardness at higher density, and that relate the hardness of k-SUM to that of k-MSUM. Below, we say that an algorithm is a (T, ε)-adversary against a problem when it solves this problem within expected runtime T and outputs a correct solution with probability at least ε.

Lemma 6.2 (Theorem 4.1 in [DKK22])**.** *Let M, N be integers such that $\sqrt{N} \leq M \leq N$, and let k be a constant. Assume that there exists an algorithm that (T, ε)-solves the $(M, N^{1/k})$-kSUM problem. Then there exists a (T', ε')-adversary against the $(N, N^{1/k})$-kSUM problem with $T' \leq (N/M) \cdot (T + \tilde{O}(N^{1/k}))$ and $\varepsilon' = \Omega(\varepsilon^4 / \log^2 M)$.*

In other words, for constant k,[5] an algorithm with a large success probability against kSUM at high densities implies an algorithm (with a larger runtime and a smaller success probability) against kSUM at density 1.

Lemma 6.3 (Lemma 4.5 in [DKK22])**.** *Let M, N be integers such that $M \leq 2N + 1$ and let k be a constant. Assume that there exists an algorithm that (T, ε)-solves the (M, r)-kMSUM problem. Then there exists a (T', ε')-adversary against the (N, r)-kSUM problem with $T' = T + \tilde{O}(r)$ and $\varepsilon' = \Omega(\varepsilon \cdot (M/N)^k)$.*

In other words, an algorithm against kMSUM yields an algorithm against kSUM with similar runtime and success probability (in the regime where $M/N = \Omega(1)$).

[5] The work of Dinur et al. [DKK22] actually handles general values of k at the cost of a $k^{O(k)}$ loss in success probability, and additional k-dependent multiplicative terms in the $\tilde{O}(N^{1/k})$ runtime overhead.

6.2 Reducing k-MMUL to k-SUM at Density 1

We consider the average-case hardness of the (q,t)-kMMUL problem (Security Game 3). A similar problem was recently considered in [AHY25]. We put forth the following conjecture:

Conjecture 6.4. *Let $k \in \{3,4\}$. Any algorithm $\mathcal{A}$ that solves the (q,t)-kMMUL problem with constant probability (over the random choice of L and its own random coins) has expected running time at least $q^{1/(k-1)-o(1)}$.*

We provide strong support for this conjecture by reducing it to the strengthened average-case k-SUM conjecture, Conjecture 2.5:

Lemma 6.5. *Conjecture 6.4 is implied by Conjecture 2.5 for $k \in \{3,4\}$.*

Concretely, we show the following sequence of reductions:

$$\begin{aligned}
&\exists(q^{1/(k-1)-\delta},\varepsilon)\text{-adversary against } (q, r = q^{1/(k-1)-\delta+o(1)})\text{-kMMUL}\\
\xRightarrow{(1)}&\exists(q^{1/(k-1)-\delta+o(1)},\varepsilon)\text{-adversary against } (q-1,r)\text{-kMSUM}\\
\xRightarrow{(2)}&\exists(q^{1/(k-1)-\delta+o(1)},\Omega(\varepsilon))\text{-adversary against } (q,r)\text{-kSUM}\\
\xRightarrow{(3)}&\exists(q^{\lceil k/2\rceil/k-\gamma},\Omega(\varepsilon^4/\log^2 q))\text{-adversary against } (N,N^{1/k})\text{-kSUM}\\
&\text{with } N = q^{k/(k-1)-k\cdot\delta+o(1)} \text{ and } \gamma=\gamma(k) \text{ with } \gamma(3),\gamma(4)>0\\
\xRightarrow{(4)}&\exists(q^{\lceil k/2\rceil/k-\gamma},\Omega(\varepsilon^4))\text{-adversary against } (N,\log^2 N\cdot N^{1/k})\text{-kSUM}.
\end{aligned}$$

Step 1 reduces kMMUL (over $\mathbb{Z}_q$) to kMSUM (over $\mathbb{Z}_{q-1}$) by moving sums over $\mathbb{Z}_{q-1}$ to products over $\mathbb{Z}_q$ via modular exponentiation. Step 2 relies on Lemma 6.3, and Step 3 on Lemma 6.2. Step 4 relies on a straightforward partitioning algorithm.

6.2.1 Step 1.

Let $\mathcal{A}$ denote a (T,ε)-adversary against the $(q, r = q^{1/(k-1)-\delta+o(1)})$-kMMUL problem (Security Game 3). Assume towards contradiction that $T = q^{1/(k-1)-\delta}$ for some $\delta < 1/(k-1)$ and $\delta, \varepsilon = \Omega(1)$. Consider the $(q-1,r)$-kMSUM problem below:

Security Game $(q-1,r)$-kMSUM problem

Parameters. A 2ℓ-bit prime q, a list size r.

Challenge. The challenger sends $L \leftarrow \mathbb{Z}_{q-1}^r$ to $\mathcal{A}$.

Output. $\mathcal{A}$ outputs a k-tuple $(i_1,\cdots,i_k) \in [r]^k$. The challenger outputs 1 if $L[i_1]+\cdots+L[i_k] = 0 \bmod q-1$, and 0 otherwise.

Claim 6.6. *There exists a $(T_1 := O(q^{1/(k-1)-\delta}\cdot\log q), \varepsilon_1 := \varepsilon - q^{\delta-1/(k-1)})$-adversary $\mathcal{A}_1$ against the $(q-1,r)$-kMSUM problem.*

In the above claim, we assume a unit cost for arithmetic operations (additions, multiplications) modulo q or $q-1$, and to read (from the input tape) or write (on the output tape) an element of $\mathbb{Z}_q$ or $\mathbb{Z}_{q-1}$.

Proof. We sketch the (straightforward) reduction: upon receiving the list $L \in \mathbb{Z}_{q-1}^r$, fix a generator g of $\mathbb{Z}_q^*$ and set $L'[j] = g^{L[j]} \bmod q$ for $j = 1$ to r. Forward L' to $\mathcal{A}$. Finally, forward $\mathcal{A}$'s answer $(i_1, \cdots, i_k)$ to the challenger.
It is immediate that if $(i_1, \cdots, i_k)$ is a valid solution to kMMUL with respect to the challenge L', then $(i_1, \cdots, i_k)$ is a valid solution to kMSUM with respect to the challenge L, and that the reduction works in time $T + T \cdot O(\log q)$ (where the constant in the $O(\cdot)$ is about 1.5 using the standard square-and-multiply algorithm over $\mathbb{Z}_q^*$). It remains to bound the probability that $(i_1, \cdots, i_k)$ is a valid solution to kMMUL. Observe that since L is uniform over $\mathbb{Z}_{q-1}^r$, it holds that L' is uniform over $(\mathbb{Z}_q^*)^r$, which (by a straightforward union bound) is r/q-close to the uniform distribution over $\mathbb{Z}_q^r$ in statistical distance. Since $r = q^{1/(k-1)-\delta+o(1)}$, the winning probability of $\mathcal{A}$ degrades by at most $q^{1/(k-1)-\delta}/q = o(1)$ when receiving a challenge from $(\mathbb{Z}_q^*)^r$ instead of $\mathbb{Z}_q^r$. □

We note that the converse direction also holds: the reduction works by computing the discrete logarithm of all entries in the lists. Since solving the discrete logarithm over $\mathbb{Z}_q$ can be done in time $\approx q^{(\ln\ln q/\ln q)^{2/3}} = q^{o(1)}$, a $q^{1/(k-1)-\delta}$-time adversary against kMSUM yields a $q^{1/(k-1)-\delta+o(1)}$-time adversary against kMMUL. This converse reduction comes with worse concrete parameters due to the cost of solving the discrete logarithm, but this is known to be inherent (in the sense that any reduction must solve the discrete logarithm), see [AHY25].

6.2.2 Step 2.

Let $\mathcal{A}_1$ be a (T_1, ε_1)-adversary against the $(q-1, r)$-kMSUM problem with $T_1 = O(q^{1/(k-1)-\delta} \cdot \log q)$, $\varepsilon_1 = \varepsilon - o(1)$, and $r = O(q^{1/(k-1)-\delta} \cdot \log q)$.

Claim 6.7. *There exists a $(T_2 := T_1 + \tilde{O}(r), \varepsilon_2 := \Omega(\varepsilon_1))$-adversary $\mathcal{A}_2$ against the (q, r)-kSUM problem (Definition 2.3).*

The claim is an immediate application of Lemma 6.3 using $M = q - 1$ and $N = q$ (note that $(M/N)^k \approx 1 - k/q \approx 1$).

6.2.3 Step 3.

Let $\mathcal{A}_2$ be a (T_2, ε_2)-adversary against the (q, r)-kSUM problem with $T_2 = O(q^{1/(k-1)-\delta} \cdot \log q)$, $\varepsilon_2 = \Omega(\varepsilon)$, and $r = O(q^{1/(k-1)-\delta} \cdot \log q)$.

Claim 6.8. *Let $N := r^k$. There exists a $(T_3 := \tilde{O}(q^{2/(k-1)-(k+1)\delta}), \varepsilon_3 := \Omega(\varepsilon_2^4/\log^2 q))$-adversary $\mathcal{A}_3$ against the $(N, N^{1/k})$-kSUM problem (Definition 2.3).*

The claim is an immediate application of Lemma 6.2 using $M := q$ and $N := r^k$. Observe that $N = \tilde{O}(q^{k/(k-1)-k\delta})$, hence $\sqrt{N} = \tilde{O}(q^{k/(2k-2)-k\delta/2}) \le M \le N$ since $k \le 2k-2$ for $k \ge 2$. Furthermore, $N/M = \tilde{O}(q^{1/(k-1)-k\delta})$, hence

- $T_3 = (T_2 + \tilde{O}(q^{1/(k-1)-\delta})) \cdot \tilde{O}(q^{1/(k-1)-k\delta}) = \tilde{O}(q^{2/(k-1)-(k+1)\delta})$
- $\varepsilon_3 = \Omega(\varepsilon_2^4/\log^2 M) = \Omega(\varepsilon^4/\log^2 q)$

Now, we observe that

- For $k = 3$ (hence $\lceil k/2 \rceil / k = 2/3$), $T_3 = \tilde{O}(q^{1-4\delta}) = \tilde{O}(N^{(1-4\delta)/(3/2-3\delta)}) = \tilde{O}(N^{2/3-2\delta/(3/2-3\delta)})$, and $2\delta/(3/2-3\delta) > 0$ since $0 < \delta < 1/(k-1) = 1/2$
- For $k = 4$ (hence $\lceil k/2 \rceil / k = 1/2$), $T_3 = \tilde{O}(q^{2/3-5\delta}) = \tilde{O}(N^{(2/3-5\delta)/(4/3-4\delta)}) = \tilde{O}(N^{1/2-9\delta/(4-12\delta)})$, and $9\delta/(4-12\delta) > 0$ since $0 < \delta < 1/(k-1) = 1/3$.

Finally, $\log^2 q = O(\log^2 N)$. Therefore, we get a contradiction to Conjecture 2.5.

6.2.4 Step 4.

For completeness, we note that the $\Omega(1/\log^2 q)$ success probability can be amplified to $\Omega(1)$ at the cost of solving the problem only at a slightly larger density $\Delta = 1 + \frac{2 \log \log N}{\log N}$. The reduction receives a larger list L of size $\log^2 N \cdot N^{1/k}$, partitions it into $\log^2 N$ lists $(L_1, \cdots, L_{\log^2 N})$, and runs the solver on each list. Note that the lists L_i have size $N^{1/k}$ and are uniform and independent; therefore, the solver succeeds with independent probability $\Omega(1/\log^2 N)$ on each instance. A straightforward calculation shows that with $1 - (1 - \Omega(1/\log^2 N))^{\log^2 N} = \Omega(1)$ probability, one of the $\log^2 N$ runs succeeds.

AI Disclosure. Generative AI tools were used to produce preliminary drafts of some material, to proofread and edit the text, and to review technical claims for correctness. In particular, the observation that the reduction can be implemented in linear time (Appendix B) was pointed out by an AI tool during this review. The final version was written by the authors, who take full responsibility for its content.

References

[ABCGJM24] Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. *Two-Round 2PC ECDSA at the Cost of 1 OLE*. Cryptology ePrint Archive, Report 2024/1950. 2024. URL: https://eprint.iacr.org/2024/1950 (cit. on pp. 3, 4, 7–9, 11, 15, 30).

[AHY25] Josh Alman, Yizhi Huang, and Kevin Yeo. "Fine-Grained Complexity in a World Without Cryptography". In: *EUROCRYPT 2025, Part VII*. LNCS. Springer, Cham, June 2025, pp. 375–405. DOI: 10.1007/978-3-031-91098-2_14 (cit. on pp. 5, 12, 22, 25, 26).

[ASSVV24] Shweta Agrawal, Sagnik Saha, Nikolaj I. Schwartzbach, Akhil Vanukuri, and Prashant Nalini Vasudevan. "k-SUM in the Sparse Regime: Complexity and Applications". In: *CRYPTO 2024, Part II*. LNCS. Springer, Cham, Aug. 2024, pp. 315–351. DOI: 10.1007/978-3-031-68379-4_10 (cit. on pp. 5, 12, 22).

[AV26] Josh Alman and Virginia Vassilevska Williams. *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*. Version 1, October 5, 2026. 2026. arXiv: 2610.06783 [cs.DS]. URL: https://arxiv.org/abs/2610.06783v1 (cit. on pp. 5, 13).

[Blo] *Presignatures*. Builder Vault TSM Docs, v72. Blockdaemon. 2025. URL: https://builder-vault-tsm.docs.blockdaemon.com/docs/presignatures (visited on 10/27/2025) (cit. on p. 3).

[BSDV21] Zvika Brakerski, Noah Stephens-Davidowitz, and Vinod Vaikuntanathan. "On the Hardness of Average-Case k-SUM". In: *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2021)*. Ed. by Mary Wootters and Laura Sanità. Vol. 207. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 29:1–29:19. ISBN: 978-3-95977-207-5. DOI: 10.4230/LIPIcs.APPROX/RANDOM.2021.29. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.APPROX/RANDOM.2021.29 (cit. on pp. 5, 12).

[CMP20] Ran Canetti, Nikolaos Makriyannis, and Udi Peled. *UC Non-Interactive, Proactive, Threshold ECDSA*. Cryptology ePrint Archive, Report 2020/492. 2020. URL: https://eprint.iacr.org/2020/492 (cit. on pp. 3, 4, 7).

[Coi21] Coinbase. *kryptology – Secure Two-party Threshold ECDSA from ECDSA Assumptions (DKLs18)*. https://github.com/coinbase/kryptology/tree/master/pkg/tecdsa/dkls/v1. Available at https://github.com/coinbase/kryptology/tree/master/pkg/tecdsa/dkls/v1. This public library has been archived and is no longer supported. 2021 (cit. on p. 4).

[Coi25] Coinbase. *cb-mpc: Coinbase MPC Library*. https://github.com/coinbase/cb-mpc. GitHub repository. MIT License. 2025. (Visited on 10/24/2025) (cit. on p. 3).

[com25] Private communication. 2025 (cit. on p. 4).

[DJNPØ20] Ivan Damgård, Thomas Pelle Jakobsen, Jesper Buus Nielsen, Jakob Illeborg Pagter, and Michael Bæksvang Østergaard. "Fast Threshold ECDSA with Honest Majority". In: *SCN 20*. Ed. by Clemente Galdi and Vladimir Kolesnikov. Vol. 12238. LNCS. Springer, Cham, Sept. 2020, pp. 382–400. DOI: 10.1007/978-3-030-57990-6_19 (cit. on pp. 3, 7).

[DKK22] Itai Dinur, Nathan Keller, and Ohad Klein. "Fine-Grained Cryptanalysis: Tight Conditional Bounds for Dense k-SUM and k-XOR". In: *62nd FOCS*. IEEE Computer Society Press, Feb. 2022, pp. 80–91. DOI: 10.1109/FOCS52979.2021.00017 (cit. on pp. 5, 12, 13, 24).

[Dkl] *DKLS.info: Resources for Threshold ECDSA (DKLS) Schemes*. https://dkls.info/. Website collecting references and materials on DKLS schemes. 2025. (Visited on 10/24/2025) (cit. on p. 3).

[DKLs18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. "Secure Two-party Threshold ECDSA from ECDSA Assumptions". In: *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018, pp. 980–997. DOI: 10.1109/SP.2018.00036 (cit. on pp. 3–8, 11).

[DKLs24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and abhi shelat. "Threshold ECDSA in Three Rounds". In: *2024 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2024, pp. 3053–3071. DOI: 10.1109/SP54263.2024.00178 (cit. on p. 6).

[DLSVW25] Mina Dalirrooyfard, Andrea Lincoln, Barna Saha, and Virginia Vassilevska Williams. "Average-Case Hardness of Parity Problems: Orthogonal Vectors, k-SUM and More". In: *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. 2025, pp. 4613–4643. DOI: 10.1137/1.9781611978322.158. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611978322.158. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611978322.158 (cit. on pp. 5, 12).

[DOKSS20] Anders P. K. Dalskov, Claudio Orlandi, Marcel Keller, Kris Shrishak, and Haya Shulman. "Securing DNSSEC Keys via Threshold ECDSA from Generic MPC". In: *ESORICS 2020, Part II*. Ed. by Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider. Vol. 12309. LNCS. Springer, Cham, Sept. 2020, pp. 654–673. DOI: 10.1007/978-3-030-59013-0_32 (cit. on pp. 3, 7).

[Dyn] Dynamic. *Our Embedded Wallets: Overview*. Dynamic Documentation. URL: https://www.dynamic.xyz/docs/wallets/embedded-wallets/mpc/overview (visited on 10/24/2025) (cit. on p. 3).

[Fir] Fireblocks. *What is Wallet-as-a-Service? Features, Benefits & Use Cases*. Fireblocks Glossary. URL: https://www.fireblocks.com/glossary/wallet-as-a-service/ (visited on 10/24/2025) (cit. on p. 3).

[Fir25] Fireblocks. *mpc-lib: Fireblocks-MPC library*. https://github.com/fireblocks/mpc-lib. GitHub repository. GPL-3.0 license. 2025. (Visited on 10/24/2025) (cit. on pp. 3–8).

[GG20] Rosario Gennaro and Steven Goldfeder. *One Round Threshold ECDSA with Identifiable Abort*. Cryptology ePrint Archive, Report 2020/540. 2020. URL: https://eprint.iacr.org/2020/540 (cit. on pp. 3, 7).

[GS22] Jens Groth and Victor Shoup. "On the Security of ECDSA with Additive Key Derivation and Presignatures". In: *EUROCRYPT 2022, Part I*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13275. LNCS. Springer, Cham, 2022, pp. 365–396. DOI: 10.1007/978-3-031-06944-4_13 (cit. on pp. 3, 4, 6–9, 11, 12, 15, 21).

[JKL24] Antoine Joux, Hunter Kippen, and Julian Loss. *A Concrete Analysis of Wagner's k-List Algorithm over $\mathbb{Z}_p$*. Cryptology ePrint Archive, Report 2024/282. 2024. URL: https://eprint.iacr.org/2024/282 (cit. on p. 6).

[LLW19] Rio LaVigne, Andrea Lincoln, and Virginia Vassilevska Williams. "Public-Key Cryptography in the Fine-Grained Setting". In: *CRYPTO 2019, Part III*. Ed. by Alexandra Boldyreva and Daniele Micciancio. Vol. 11694. LNCS. Springer, Cham, Aug. 2019, pp. 605–635. DOI: 10.1007/978-3-030-26954-8_20 (cit. on pp. 5, 12).

[Pkc] *PKCS #11 Cryptographic Token Interface Current Mechanisms, Version 2.40*. CKM_ECDSA "ECDSA without hashing"; see signature encoding reference in S2.3.1. OASIS. 2015. URL: https://docs.oasis-open.org/pkcs11/pkcs11-curr/v2.40/os/pkcs11-curr-v2.40-os.html (cit. on p. 4).

[Pro25] Lockness Project. *Lockness cryptography libraries*. https://github.com/LFDT-Lockness. GitHub organization under LF Decentralized Trust. 2025. (Visited on 10/24/2025) (cit. on p. 3).

[Qui23a] Quilibrium. https://quilibrium.com/. Main protocol uses 2-party ECDSA signing, see https://docs.quilibrium.com/docs/run-node/qclient/commands/compute. The main protocol implementation is the updated DKLs18 on the nekryptology repository. 2023 (cit. on pp. 4, 6).

[Qui23b] Quilibrium. *nekryptology – implementation of DKLs18*. https://github.com/QuilibriumNetwork/monorepo/tree/develop/nekryptology. Available at https://github.com/QuilibriumNetwork/monorepo/tree/develop/nekryptology. Fork of kryptology: removes GG20 and updates DKLs18 with KOS15. 2023 (cit. on pp. 4, 6).

[Set15] Seth Pettie. *Higher Lower Bounds from the 3SUM Conjecture*. Fine-Grained Complexity and Algorithm Design Workshop at the Simons Institute. Available at https://www.youtube.com/watch?v=OkagNffn7KQ. 2015 (cit. on pp. 5, 12).

[Sho97] Victor Shoup. "Lower Bounds for Discrete Logarithms and Related Problems". In: *EUROCRYPT'97*. Ed. by Walter Fumy. Vol. 1233. LNCS. Springer, Berlin, Heidelberg, May 1997, pp. 256–266. DOI: 10.1007/3-540-69053-0_18 (cit. on p. 4).

[Tau21] TaurusHQ. *multi-party-sig: DKLS18 Protocols*. https://github.com/taurusgroup/multi-party-sig/tree/main/protocols/doerner. Available at https://github.com/taurusgroup/multi-party-sig/tree/main/protocols/doerner. 2021 (cit. on p. 4).

[Wag02] David Wagner. "A Generalized Birthday Problem". In: *CRYPTO 2002*. Ed. by Moti Yung. Vol. 2442. LNCS. Springer, Berlin, Heidelberg, Aug. 2002, pp. 288–303. DOI: 10.1007/3-540-45708-9_19 (cit. on pp. 6, 21).

[Wui12] Pieter Wuille. *BIP 32: Hierarchical Deterministic Wallets*. https://bips.dev/32/. Key derivation: CKDpriv/CKDpub with HMAC-SHA512; see "Specification: Key derivation". 2012 (cit. on pp. 4, 8).

A GGM Forgery to Lazy Evaluation Forgery Reduction

In this section, we show that the symbolic evaluation from Algorithm 1 is indistinguishable from the GGM up to an error term of $O(t^2/q)$ with a very small constant, where t denotes the number of queries of the attacker. Namely, we prove the following theorem. The proof herein is essentially an adaptation of the analogous proof of Adjedj et al. [ABCGJM24], which we follow closely.

Theorem A.1. *For an algorithm* D *interacting with the group oracle in the GGM or the lazy symbolic evaluation from Algorithm 1 making at most* t *queries to the oracle, let* $\mathrm{out}_{\mathsf{D}}$ *and* $\mathrm{out}'_{\mathsf{D}}$ *denote the output of* D *when interacting with the respective oracle. Then, for every* D*,*

$$\left|\Pr_{\mathsf{D}}[\mathrm{out}_{\mathsf{D}} = 1] - \Pr_{\mathsf{D}}[\mathrm{out}'_{\mathsf{D}} = 1]\right| \leq \frac{(t+5)(t+4)}{q} + 2 \cdot \frac{(t+4)(2t+9)+2t}{q-1}$$

Proof. The above is a corollary of Propositions A.2, A.3 and A.5 stated further below. □

A.1 Lazy Evaluation 1

Let $\mathsf{SymbLazyEval}_2$ denote Algorithm 1 and let $\mathsf{LazyEval}_1$ denote the group oracle defined by the following operations. This oracle corresponds to the standard lazy evaluation oracle of the GGM, suitably modified for our use case.

Algorithm $\mathsf{LazyEval}_1$

1. Initialization.
   (a) $\pi \leftarrow \{(0, \mathbb{1})\}$
   (b) Sample $g \xleftarrow{\$} \boldsymbol{G} \setminus \{\mathbb{1}\}$ and set $\pi \leftarrow \pi \cup \{(1, g), (-1, g^{-1})\}$.
   (c) Sample $x \xleftarrow{\$} \mathbb{F}_q^*$ and $X \xleftarrow{\$} \boldsymbol{G}$. Set $\pi \leftarrow \pi \cup \{(x, X), (-x, X^{-1})\}$.
   (d) Set $\boldsymbol{R} \leftarrow \varnothing$ (the presignatures set)
   (e) Return (g, X).
2. Hash query. On input $\text{msg} \in \{0,1\}^*$, do:
   (a) If $\text{msg} \notin \text{domain}(\eta)$
      i. Sample $h \xleftarrow{\$} \mathbb{F}_q$.
      ii. Set $\eta \leftarrow \eta \cup \{(\text{msg}, h)\}$.
   (b) Return $\eta(\text{msg})$
3. Group query. On input $i \in \mathbb{F}_q$, do:
   (a) If $i \notin \text{domain}(\pi)$
      i. Sample $P \xleftarrow{\$} \boldsymbol{G}$ (if $P \in \text{range}(\pi)$, repeat this step)
      ii. Set $\pi \leftarrow \pi \cup \{(i, P), (-i, P^{-1})\}$.
   (b) Return $\pi(i)$
4. Group addition. On input $(P_1, P_2) \in \boldsymbol{G}^2$, do:
   (a) For $j \in \{1, 2\}$, if $P_j \notin \text{range}(\pi)$:
      i. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$ (if $i \in \text{domain}(\pi)$, repeat this step)
      ii. Set $\pi \leftarrow \pi \cup \{(i, P_j), (-i, P_j^{-1})\}$.
   (b) Return the group query output on input $\pi^{-1}(P_1) + \pi^{-1}(P_2)$.
5. Presignatures. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$, and do:
   (a) If $i \notin \text{domain}(\pi)$, invoke the group query on input i to obtain R.
   (b) Return $R = \pi(i)$ and set $\boldsymbol{R} \leftarrow \boldsymbol{R} \cup \{R\}$.
6. Modifier. On input $(R, m, \text{id}) \in \boldsymbol{R} \times \mathbb{F}_q^* \times \{0,1\}^*$, do:
   (a) If $(R, m, \text{id}) \notin \text{domain}(\lambda)$

i. Sample $\mu \xleftarrow{\$} \mathbb{F}_q$
ii. Invoke the group query on input $\mu \cdot \pi^{-1}(R)$
iii. Set $\lambda \leftarrow \lambda \cup ((R, m, \mathrm{id}), \mu)$.

(b) Return $\lambda(R, m, \mathrm{id})$

7. Signature. On input $(R, m, \mathrm{id}) \in \mathrm{domain}(\lambda)$, do:

(a) Let $\mu = \lambda(R, m, \mathrm{id})$ and $\hat{R} = \pi(\mu \cdot \pi^{-1}(R))$
(b) Set $\hat{r} = \mathcal{X}(\hat{R})$ and remove all records of R from $\boldsymbol{R}$ and λ
(c) Return $(\hat{r}, s)$ for $s \leftarrow (\pi^{-1}(\hat{R}))^{-1}(m + \hat{r}x) \bmod q$.
(If $m + \hat{r}x = 0$ or $\hat{r} = 0$, return fail)

A.2 Lazy Evaluation 2

Let $\mathsf{LazyEval}_2$ denote the group oracle defined by the following operations. Changes with respect to $\mathsf{LazyEval}_1$ are highlighted.

Algorithm $\mathsf{LazyEval}_2$

1. Initialization.

(a) $\pi \leftarrow \{(0, \mathbb{1})\}$
(b) Sample $g \xleftarrow{\$} \boldsymbol{G} \setminus \{\mathbb{1}\}$ and set $\pi \leftarrow \pi \cup \{(1, g), (-1, g^{-1})\}$.
(c) Sample $x \xleftarrow{\$} \mathbb{F}_q^*$ and $X \xleftarrow{\$} \boldsymbol{G}$. (If $X \in \mathrm{range}(\pi)$, abort)
(d) Set $\pi \leftarrow \pi \cup \{(x, X), (-x, X^{-1})\}$.
(e) Set $\boldsymbol{R} \leftarrow \varnothing$ (the presignatures set)
(f) For all $P \in \boldsymbol{G}$ such that $\mathcal{X}(P) = 0$, execute the group addition query on input $(P, \mathbb{1})$.
(g) Return (g, X)

2. Hash query. On input $\mathrm{msg} \in \{0,1\}^*$, do:

(a) If $\mathrm{msg} \notin \mathrm{domain}(\eta)$
i. Sample $h \xleftarrow{\$} \mathbb{F}_q$.
ii. Set $\eta \leftarrow \eta \cup \{(\mathrm{msg}, h)\}$.
(b) Return $\eta(\mathrm{msg})$

3. Group query. On input $i \in \mathbb{F}_q$, do:

(a) If $i \notin \mathrm{domain}(\pi)$
i. Sample $P \xleftarrow{\$} \boldsymbol{G}$. (If $P \in \mathrm{range}(\pi)$, abort)
ii. Set $\pi \leftarrow \pi \cup \{(i, P), (-i, P^{-1})\}$.

(b) Return $\pi(i)$

4. Group addition. On input $(P_1, P_2) \in \boldsymbol{G}^2$, do:
   (a) For $j \in \{1, 2\}$, if $P_j \notin \text{range}(\pi)$:
      i. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$ (if $i \in \text{domain}(\pi)$, abort)
      ii. Set $\pi \leftarrow \pi \cup \{(i, P_j), (-i, P_j^{-1})\}$.
   (b) Return the group query output on input $\pi^{-1}(P_1) + \pi^{-1}(P_2)$.
5. Presignatures. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$ (If $i \in \text{domain}(\pi)$, abort), and do:
   (a) Invoke the group query on input i.
   (b) Return $R = \pi(i)$ and set $\boldsymbol{R} \leftarrow \boldsymbol{R} \cup \{R\}$.
6. Modifier. On input $(R, m, \text{id}) \in \boldsymbol{R} \times \mathbb{F}_q^* \times \{0,1\}^*$, do:
   (a) If $(R, m, \text{id}) \notin \text{domain}(\lambda)$
      i. Sample $\mu \xleftarrow{\$} \mathbb{F}_q$ (If $\mu \cdot \pi^{-1}(R) \in \text{domain}(\pi)$, abort)
      ii. Invoke the group query on input $\mu \cdot \pi^{-1}(R)$
      iii. Set $\lambda \leftarrow \lambda \cup ((R, m, \text{id}), \mu)$.
   (b) Return $\lambda(R, m, \text{id})$
7. Signature. On input $(R, m, \text{id}) \in \text{domain}(\lambda)$, do:
   (a) Let $\mu = \lambda(R, m, \text{id})$ and $\hat{R} = \pi(\mu\pi^{-1}(R))$
   (b) Set $\hat{r} = \mathcal{X}(\hat{R})$ and remove all records of R from $\boldsymbol{R}$ and λ
   (c) Return $(\hat{r}, s)$ for $s \leftarrow (\pi^{-1}(\hat{R}))^{-1}(m + \hat{r}x) \bmod q$.
   (If $m + \hat{r}x = 0$, return fail)

Proposition A.2. *For an algorithm* D *interacting with* $\mathsf{LazyEval}_1$ *or* $\mathsf{LazyEval}_2$ *making at most* t *queries to the group oracle, let* $\text{out}_{\mathsf{D}}^{\mathsf{LazyEval}_i}$ *denote the output of* D *when interacting with* $\mathsf{LazyEval}_i$. *Then, for every* D,

$$\left| \Pr_{\mathsf{D}}\left[\text{out}_{\mathsf{D}}^{\mathsf{LazyEval}_1} = 1\right] - \Pr_{\mathsf{D}}\left[\text{out}_{\mathsf{D}}^{\mathsf{LazyEval}_2} = 1\right] \right| \leq \frac{(t+5)(t+4)}{q}$$

Proof. Note that the two experiments are distinguishable only if $\mathsf{LazyEval}_2$ aborts. At any given query, $\mathsf{LazyEval}_2$ aborts with probability $\frac{|\pi|}{q}$. After initialization, we have $|\pi| \leq 9$. Since each query adds at most two elements to π, the size of π at the i-th query is at most $9 + 2(i-1)$. Therefore,

the total probability that $\mathsf{LazyEval}_2$ aborts over t queries is at most

$$\sum_{i=1}^{t} \frac{9+2(i-1)}{q} = \frac{t}{2} \cdot \frac{9+(2t+7)}{q}$$
$$= \frac{t(t+8)}{q} \leq \frac{(t+5)(t+4)}{q}$$

□

A.3 Symbolic Evaluation 1

Let $\mathsf{SymbLazyEval}_1$ denote the group oracle defined by the following operations. Changes with respect to $\mathsf{LazyEval}_2$ are highlighted.

Algorithm $\mathsf{SymbLazyEval}_1$

1. Initialization.
   (a) $\pi \leftarrow \{(0, \mathbb{1})\}$
   (b) Sample $g \xleftarrow{\$} \boldsymbol{G} \setminus \{\mathbb{1}\}$ and set $\pi \leftarrow \pi \cup \{(1, g), (-1, g^{-1})\}$.
   (c) Sample $x \xleftarrow{\$} \mathbb{F}_q^*$ and $X \xleftarrow{\$} \boldsymbol{G}$. (If $X \in \mathrm{range}(\pi)$, abort)
   (d) Set $\rho = \{(x, X), (-x, X^{-1})\}$and $\pi \leftarrow \pi \cup \{(\mathfrak{R}_1, X), (-\mathfrak{R}_1, X^{-1})\}$.
   (e) Set $\boldsymbol{V} \leftarrow \{\mathfrak{R}_1\}$ and $\boldsymbol{R} \leftarrow \varnothing$ (the presignatures set) and $c = 1$.
   (f) For all $P \in \boldsymbol{G}$ such that $\mathcal{X}(P) = 0$, execute the group addition query on input $(P, \mathbb{1})$.
   (g) Return (g, X)
2. Hash query. On input $\mathrm{msg} \in \{0,1\}^*$, do:
   (a) If $\mathrm{msg} \notin \mathrm{domain}(\eta)$
      i. Sample $h \xleftarrow{\$} \mathbb{F}_q$.
      ii. Set $\eta \leftarrow \eta \cup \{(\mathrm{msg}, h)\}$.
   (b) Return $\eta(\mathrm{msg})$
3. Group query. On input $i \in \mathbb{F}_q$, do:
   (a) If $i \notin \mathrm{domain}(\pi)$
      i. Sample $P \xleftarrow{\$} \boldsymbol{G}$. (If $P \in \mathrm{range}(\pi)$, abort)
      ii. Set $\pi \leftarrow \pi \cup \{(i, P), (-i, P^{-1})\}$.
   (b) Return $\pi(i)$
4. Group addition. On input $(P_1, P_2) \in \boldsymbol{G}^2$, do:
   (a) For $j \in \{1, 2\}$, if $P_j \notin \mathrm{range}(\pi)$:
      i. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$. (If $i \in \mathrm{domain}(\pi)$, abort)

ii. Set $\pi \leftarrow \pi \cup \{(i, P_j), (-i, P_j^{-1})\}$.

(b) Return the group query output on input $\pi^{-1}(P_1) + \pi^{-1}(P_2)$.

5. Presignatures. Set $c \leftarrow c + 1$, and do:

   (a) Sample $R \xleftarrow{\$} \boldsymbol{G}$. (If $R \in \text{range}(\pi)$, abort).

   (b) Sample $i \xleftarrow{\$} \mathbb{F}_q^*$ and set $\rho \leftarrow \rho \cup \{(i, R), (-i, R^{-1})\}$.

   (c) Set $\pi \leftarrow \pi \cup \{(\mathfrak{R}_c, R), (-\mathfrak{R}_c, R^{-1})\}$, $\boldsymbol{V} \leftarrow \boldsymbol{V} \cup \{\mathfrak{R}_c\}$ and $\boldsymbol{R} \leftarrow \boldsymbol{R} \cup \{R\}$.

   (d) Return R.

6. Modifier. On input $(R, m, \text{id}) \in \boldsymbol{R} \times \mathbb{F}_q^* \times \{0,1\}^*$, do:

   (a) If $(R, m, \text{id}) \notin \text{domain}(\lambda)$

      i. Sample $\mu \xleftarrow{\$} \mathbb{F}_q$. (If $\mu \cdot \pi^{-1}(R) \in \text{domain}(\pi)$, abort)

      ii. Obtain $\hat{R}$ by invoking the group query on input $\mu \cdot \pi^{-1}(R)$.

      iii. Set $\lambda \leftarrow \lambda \cup ((R, m, \text{id}), \mu)$ and $\rho \leftarrow \rho \cup \{(\mu \cdot \rho^{-1}(R), \hat{R})\}$.

   (b) Return $\lambda(R, m, \text{id})$

7. Signature. On input $(R, m, \text{id}) \in \text{domain}(\lambda)$, do:

   (a) Let $\mu = \lambda(R, m, \text{id})$, $\hat{R} = \pi(\mu \cdot \pi^{-1}(R))$ and $\hat{r} = \mathcal{X}(\hat{R})$.

   (b) Set $s = (\rho^{-1}(\hat{R}))^{-1}(m + \hat{r}x)$ if $m + \hat{r}x \neq 0$ and $s \xleftarrow{\$} \mathbb{F}_q^*$, otherwise.

   (c) Substitute $s^{-1}(m + \hat{r} \cdot \mathfrak{R}_1)$ for $\pi^{-1}(\hat{R})$ throughout $\text{domain}(\pi)$.

   Abort if any two polynomials collapse, i.e., there's a collision.

   (d) Remove all records of R and $\mathfrak{R}$ from $\boldsymbol{R}$, $\boldsymbol{V}$ and λ.

   (e) Return $(\hat{r}, s)$

Proposition A.3. *For an algorithm* D *interacting with* $\mathsf{LazyEval}_2$ *or* $\mathsf{SymbLazyEval}_1$ *making at most* t *queries to the group oracle, let* $\text{out}_{\mathsf{D}}^{\mathsf{LazyEval}_2}$ *and* $\text{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_1}$ *denote the output of* D *when interacting with the corresponding oracle. Then, for every* D,

$$\left| \Pr_{\mathsf{D}}\left[\text{out}_{\mathsf{D}}^{\mathsf{LazyEval}_2} = 1\right] - \Pr_{\mathsf{D}}\left[\text{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_1} = 1\right] \right| \leq \frac{2(t+4)(2t+9) + 4t}{q-1}$$

Proof. Notice that the two experiments are distinguishable only in the following cases:

1. It occurred that $m + \hat{r}x = 0$ in Item 7b

2. Two polynomials in $\text{domain}(\pi)$ collapsed in Item 7c

3. One of the highlighted items would have aborted according to $\mathsf{LazyEval}_2$ (because $i \notin \text{domain}(\pi)$ according to $\mathsf{SymbLazyEval}_1$ and yet $i = i_0 + \sum_{\mathfrak{R} \in \boldsymbol{V}} i_{\mathfrak{R}} \cdot \rho^{-1}(\pi(\mathfrak{R}))$ for $i_0 + \sum_{\mathfrak{R} \in \boldsymbol{V}} i_{\mathfrak{R}} \cdot \mathfrak{R} \in \text{domain}(\pi)$).

We show that the first two items occur with probability at most $4t/(q-1)$ and $(t+4)(2t+9)/(q-1)$, respectively, and, assuming Item 2 does not occur, Item 3 occurs with probability at most $(t+4)(2t+9)/(q-1)$, which immediately implies the claimed bound. The first item follows from Fact A.4 (stated further below) as each $\hat{r}$ is random. For the second item, we count how many points $P, P' \in \text{range}(\pi)$ are susceptible to collapse. Assume that when $\hat{R} = \pi(\mu \cdot \mathfrak{R}_2)$ is queried for a signature, for some $\mathfrak{R}_2 \in \boldsymbol{V}$, and P and P' admit the following expressions in domain(π):

$$\begin{cases} P = i_0 + i_1\mathfrak{R}_1 + i_2(\mu \cdot \mathfrak{R}_2) + \sum_{\mathfrak{R} \in \boldsymbol{V} \setminus \{\mathfrak{R}_1, \mathfrak{R}_2\}} i_{\mathfrak{R}} \cdot \mathfrak{R} \\ P' = i'_0 + i'_1\mathfrak{R}_1 + i'_2(\mu \cdot \mathfrak{R}_2) + \sum_{\mathfrak{R} \in \boldsymbol{V} \setminus \{\mathfrak{R}_1, \mathfrak{R}_2\}} i'_{\mathfrak{R}} \cdot \mathfrak{R} \end{cases}$$

Note that the polynomials associated with P and P' can collide only if $i_j \neq i'_j$ for $j \in \{0,1,2\}$ and $i_{\mathfrak{R}} = i'_{\mathfrak{R}}$ for all $\mathfrak{R} \in \boldsymbol{V} \setminus \{\mathfrak{R}_1, \mathfrak{R}_2\}$. In such a case, if P and P' are a potentially colliding pair but do not collide when the signature associated with $\hat{R}$ is released, they cannot collide in any subsequent signature query. Therefore, each pair $\{P, P'\}$ is counted at most once as a potentially colliding pair. Since there are $\binom{|\pi|}{2}$ such pairs, the Schwartz-Zippel lemma implies that the probability that two polynomials collide is at most

$$\binom{|\pi|}{2} \cdot \frac{1}{q-1} = \frac{(9+2t)(9+2t-1)}{2(q-1)} = \frac{(2t+9)(t+4)}{q-1}$$

To conclude, under the condition that no two polynomials collide, we apply Schwartz-Zippel once again to deduce that the probability that Item 3 occurs is at most $(2t+9)(t+4)/(q-1)$. □

Fact A.4. *Let $(\mathbb{G}, g, q)$ be the group-generator-order tuple associated with a prime-order elliptic curve, and let $a \in \mathbb{F}_q$. Then, $\Pr_{k \leftarrow \mathbb{F}_q^*}\left[\mathcal{X}(g^k) = a\right] \leq \frac{4}{q-1}$.*

A.4 Symbolic Evaluation 2

Let $\mathsf{SymbLazyEval}_2$ denote the group oracle defined by the following operations. This oracle corresponds to the lazy symbolic oracle from Algorithm 1.

Algorithm $\mathsf{SymbLazyEval}_2$

1. Initialization.
   (a) $\pi \leftarrow \{(0, \mathbb{1})\}$
   (b) Sample $g \xleftarrow{\$} \boldsymbol{G}$ and set $\pi \leftarrow \pi \cup \{(1, g), (-1, g^{-1})\}$.
   (c) Sample $X \xleftarrow{\$} \boldsymbol{G}$ and $\pi \leftarrow \pi \cup \{(\mathfrak{R}_1, X), (-\mathfrak{R}_1, X^{-1})\}$. (If $X \in \text{range}(\pi)$, abort)
   (d) Set $\boldsymbol{V} \leftarrow \{\mathfrak{R}_1\}$ and $\boldsymbol{R} \leftarrow \varnothing$ (the presignatures set) and $c = 1$.
   (e) For all $P \in \boldsymbol{G}$ such that $\mathcal{X}(P) = 0$, execute the group addition query on input $(P, \mathbb{1})$.
   (f) Return (g, X)
2. Hash query. On input msg $\in \{0,1\}^*$, do:
   (a) If msg $\notin$ domain(η)

i. Sample $h \xleftarrow{\$} \mathbb{F}_q$.

ii. Set $\eta \leftarrow \eta \cup \{(\text{msg}, h)\}$.

(b) Return $\eta(\text{msg})$

3. Group query. On input $i = i_0 + \sum_{\mathfrak{R} \in \boldsymbol{V}} i_{\mathfrak{R}} \cdot \mathfrak{R}$, do:

(a) If $i \notin \text{domain}(\pi)$

i. Sample $P \xleftarrow{\$} \boldsymbol{G}$. (If $P \in \text{range}(\pi)$, abort)

ii. Set $\pi \leftarrow \pi \cup \{(i, P), (-i, P^{-1})\}$.

(b) Return $\pi(i)$

4. Group addition. On input $(P_1, P_2) \in \boldsymbol{G}^2$, do:

(a) For $j \in \{1, 2\}$, if $P_j \notin \text{range}(\pi)$:

i. Sample $i \xleftarrow{\$} \mathbb{F}_q^*$. (If $i \in \text{domain}(\pi)$, abort)

ii. Set $\pi \leftarrow \pi \cup \{(i, P_j), (-i, P_j^{-1})\}$.

(b) Return the group query output on input $\pi^{-1}(P_1) + \pi^{-1}(P_2)$.

5. Presignatures. Set $c \leftarrow c + 1$, and do:

(a) Sample $R \xleftarrow{\$} \boldsymbol{G}$. (If $R \in \text{range}(\pi)$, abort).

(b) Set $\pi \leftarrow \pi \cup \{(\mathfrak{R}_c, R), (-\mathfrak{R}_c, R^{-1})\}$, $\boldsymbol{V} \leftarrow \boldsymbol{V} \cup \{\mathfrak{R}_c\}$ and $\boldsymbol{R} \leftarrow \boldsymbol{R} \cup \{R\}$.

(c) Return R.

6. Modifier. On input $(R, m, \text{id}) \in \boldsymbol{R} \times \mathbb{F}_q^* \times \{0,1\}^*$, do:

(a) If $(R, m, \text{id}) \notin \text{domain}(\lambda)$

i. Sample $\mu \xleftarrow{\$} \mathbb{F}_q$. (If $\mu \cdot \pi^{-1}(R) \in \text{domain}(\pi)$, abort)

ii. Invoke the group query on input $\mu \cdot \pi^{-1}(R)$.

iii. Set $\lambda \leftarrow \lambda \cup ((R, m, \text{id}), \mu)$.

(b) Return $\lambda(R, m, \text{id})$

7. Signature. On input $(R, m, \text{id}) \in \text{domain}(\lambda)$, do:

(a) Let $\mu = \lambda(R, m, \text{id})$ and $\hat{R} = \pi(\mu \cdot \pi^{-1}(R))$.

(b) Set $\hat{r} = \mathcal{X}(\hat{R})$ and sample $s \xleftarrow{\$} \mathbb{F}_q^*$.

(c) Substitute $s^{-1}(m + \hat{r} \cdot \mathfrak{R}_1)$ for $\pi^{-1}(\hat{R})$ throughout $\text{domain}(\pi)$.

Abort if any two polynomials collapse, i.e., there's a collision.

(d) Remove all records of R and $\mathfrak{R}$ from $\boldsymbol{R}$, $\boldsymbol{V}$ and λ.

(e) Return $(\hat{r}, s)$

Proposition A.5. *For an algorithm* D *interacting with* $\mathsf{SymbLazyEval}_2$ *or* $\mathsf{SymbLazyEval}_1$ *making at most* t *queries to the group oracle, let* $\mathrm{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_1}$ *and* $\mathrm{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_2}$ *denote the output of* D *when interacting with the corresponding oracle. Then, for every* D*,*

$$\left|\Pr_{\mathsf{D}}\left[\mathrm{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_2}=1\right]-\Pr_{\mathsf{D}}\left[\mathrm{out}_{\mathsf{D}}^{\mathsf{SymbLazyEval}_1}=1\right]\right|=0$$

B Linear-Time HashInv Solver

We replace the $O(tu)$ bookkeeping of Section 4 by $O(t)$ field operations and expected dictionary operations. This is needed for the fine-grained runtime claims of the 3MMUL reduction and its variants. We only treat the HashInv solver of Algorithm 3 for multiplicatively rerandomized presignatures without key derivation. This is the hardest case, since the solver programs one signature, which forces a rebuild of the dictionary. All other variants are handled analogously: in the other signing modes (plain or additively rerandomized presignatures, with or without key derivation) only the signing equation changes, and the 3MMUL, 4MMUL and AFF solvers are simpler, as they program no signature and need no rebuild. For hash inversion in the ROM alone, the original implementation suffices because its additional computation does not increase the number of hash queries.

Recalling the HashInv Solver. Let B denote the solver of Algorithm 3. It represents every group element by a formal affine function of the key variable $\mathfrak{R}_1$ and the presignature variables $\mathfrak{R}_c$, $c \geq 2$, and detects collisions between these functions. Signing presignature $\mathfrak{R}_c$ samples a fresh scalar s and substitutes the corresponding expression for $\mathfrak{R}_c$ in every stored function. Since a function may involve up to $u \leq t$ presignature variables, maintaining $O(t)$ of them costs $O(tu)$. To invert the challenge e, B guesses the query defining the forgery nonce, losing a factor $1/t$, and with equal probability programs as a function of e either its label or the signature that consumes the last presignature it involves.
Our solver $\mathsf{B}_{\mathrm{lin}}$ does not store expanded symbolic functions. Instead, it assigns each group element a *fingerprint*: a single field element that acts as a simulated discrete logarithm and is used to detect collisions. Ordinary signatures are computed from the fingerprints and leave them unchanged. Only the programmed signature needs a scalar that depends on the symbolic expression of the prospective forgery nonce, and this scalar may be inconsistent with the fingerprints; there, $\mathsf{B}_{\mathrm{lin}}$ resets the fingerprint of the presignature and rebuilds the dictionary. To obtain the expression of the nonce, $\mathsf{B}_{\mathrm{lin}}$ stores the defining function of each group element in terms of earlier ones, unexpanded, and evaluates it when needed. The evaluation and the rebuild each take $O(t)$ time and happen only once, so the total running time remains linear. In detail, $\mathsf{B}_{\mathrm{lin}}$ works as follows.

Defining Functions. Write $i_P = \pi^{-1}(P)$ for the exponent associated with label P. For each new group result, store its defining function of previously recorded exponents: for example, $C = AB$ gives $i_C := i_A + i_B$, with references to the records for A and B. Composing these definitions gives an affine function of the key and presignature variables, but we keep the definitions unexpanded. There are $O(t)$ defining operations; explicitly supplied coefficients are charged to the attacker's time. Store each consumed variable's substitution separately. All values can then be recomputed in $O(t)$ time by evaluating the variables followed by the definitions in order.

Fingerprints. Independently sample $\nu(\mathfrak{R}_c) \xleftarrow{\$} \mathbb{Z}_q$ for every new formal variable, including $\mathfrak{R}_1$. Store each function's value $\nu(i_P)$ (extending ν to affine functions by evaluation), called its *fingerprint*, and maintain dictionaries from fingerprints to labels and from labels to function definitions. These values are private to the simulator. Group queries use fingerprints for lookup, sampling labels as in B.

Ordinary Signatures. Recall that $R = \pi(\mathfrak{R}_c)$, $\hat{R} = \pi(\mu\mathfrak{R}_c)$, and $\hat{r} = \mathcal{X}(\hat{R})$. Except at the one signature specified below, set

$$A := m + \hat{r}\nu(\mathfrak{R}_1), \qquad w := \mu\nu(\mathfrak{R}_c), \qquad s := A/w, \qquad \mathfrak{R}_c \longleftarrow a_c + b_c\mathfrak{R}_1 := \frac{m + \hat{r}\mathfrak{R}_1}{\mu s}, \tag{3}$$

aborting if $A = 0$ or $w = 0$. Since $a_c + b_c\nu(\mathfrak{R}_1) = \nu(\mathfrak{R}_c)$, the substitution preserves every stored value. No dictionary update is needed. A symbolic collapse between distinct labels would require their fingerprints to have already coincided; these events are bounded below.

Tracking the Prospective Forgery Nonce. At query j_0, recover the exact coefficients of the guessed function in one backward pass through the stored definitions. Start with weight 1 for that function. In reverse order, distribute each accumulated weight to its summands, multiplying by the scalar for a scalar multiplication. At each original variable, accumulate its coefficient; fold any recorded substitution into the constant and $\mathfrak{R}_1$ coefficients. This costs $O(t)$ operations.
Write the result as $\alpha + \beta\mathfrak{R}_1 + \sum_{c\geq 2} i_c\mathfrak{R}_c$, summing over unconsumed presignatures. For $b = 0$, program the label as in Algorithm 3. For $b = 1$, store Z_0 and $\boldsymbol{S} = \{\mathfrak{R}_c : c \geq 2,\ i_c \neq 0\}$. Each ordinary signature on some $\mathfrak{R}_c \in \boldsymbol{S}$ updates $\alpha \leftarrow \alpha + i_c a_c$ and $\beta \leftarrow \beta + i_c b_c$, and removes $\mathfrak{R}_c$ from $\boldsymbol{S}$. Store the coefficients and $|\boldsymbol{S}|$ so that each update takes constant time.

The Programmed Signature. When a signature consumes the last variable in $\boldsymbol{S}$, write $Z_0 = g^\alpha X^\beta \hat{R}^\gamma$, where $\gamma = i_c/\mu$, and let $f = \mathcal{X}(Z_0)$. If $\alpha\hat{r} - \beta m \neq 0$, choose the programmed scalar from Algorithm 3:

$$s = \mathsf{ssig}(\alpha, \beta, \gamma, f, m, \hat{r}, e) = \gamma\frac{e\hat{r} - fm}{\alpha f - e\beta}, \qquad \nu(\mathfrak{R}_c) \leftarrow \frac{m + \hat{r}\nu(\mathfrak{R}_1)}{\mu s}. \tag{4}$$

If the condition fails or the scalar is undefined or zero, sample $s \xleftarrow{\$} \mathbb{Z}_q^*$ independently and use the same reset. Record the substitution, reevaluate all functions, and rebuild the dictionary, aborting on duplicate values for distinct labels. We call this signature the *programmed signature*, and write c^* for its presignature and s^* for its scalar. It is the only signature whose scalar is not computed from fingerprints; instead, the fingerprint $\nu(\mathfrak{R}_{c^*})$ is reset to match s^*.

Proposition B.1. *Let* B *denote the solver with exact symbolic bookkeeping and* $\mathsf{B}_{\mathrm{lin}}$ *the algorithm described above. Let* $n_\pi = O(t)$ *bound the number of entries of* π*. The solver* $\mathsf{B}_{\mathrm{lin}}$ *uses* $O(t)$ *additional field operations, expected dictionary operations, and storage, and satisfies*

$$\Pr[\mathsf{F}(\mathsf{B}_{\mathrm{lin}}^{\mathsf{A}}(e)) = e] \geq (1 - \varepsilon_{\mathrm{fp}})\Pr[\mathsf{F}(\mathsf{B}^{\mathsf{A}}(e)) = e], \qquad \varepsilon_{\mathrm{fp}} := \min\left\{1, \frac{2\binom{n_\pi}{2} + t}{q}\right\}.$$

The probabilities include $e \xleftarrow{\$} \mathbb{Z}_q$ *and all random choices. Both solvers make the same hash queries on corresponding executions.*

Proof. We fix a transcript of B and lower-bound the probability, over the fingerprints, that $\mathsf{B}_{\mathrm{lin}}$ produces the same transcript. There are two losses: (i) ordinary signature scalars are derived from fingerprints instead of being sampled, and (ii) two labels that B keeps distinct may receive equal fingerprints.

Setup. Fix e, j_0, b, the attacker's coins, the hash answers, and a successful, non-aborting transcript $\mathcal{T}$ of B, including the labels, modifiers, and signature scalars, together with the random choices that produce them. Only the private fingerprints of $\mathsf{B}_{\mathrm{lin}}$ remain random. Since $\mathcal{T}$ is fixed, so are the query sequence and every symbolic definition and substitution, even though the queries were adaptive. Write $x = \nu(\mathfrak{R}_1)$ for the key fingerprint and $u_c = \nu(\mathfrak{R}_c)$ for the fingerprint sampled when presignature c is created.

Loss (i): Signature Scalars. For an ordinary signature in $\mathcal{T}$ to receive its fixed scalar s, Equation (3) requires

$$u_c = \frac{m + \hat{r}x}{\mu s}.$$

We call these the *prescribed* values. Each is hit with probability $1/q$, whereas B samples s with probability $1/(q-1)$, so over the $k \le t$ ordinary signatures the probability ratio is $((q-1)/q)^k \ge 1 - t/q$. All other fingerprints are *free*, i.e., independent and uniform: x, the u_c of presignatures without an ordinary signature in $\mathcal{T}$, and u_{c^*}, since s^* is chosen without this equation. The programmed signature follows the same rule in both solvers and contributes no loss; at the reset, u_{c^*} is replaced by $u^+_{c^*} := (m + \hat{r}x)/(\mu s^*)$.

Loss (ii): Fingerprint Collisions. Condition on the prescribed values; the free fingerprints remain independent and uniform. An ordinary substitution $\mathfrak{R}_c \mapsto (m + \hat{r}\mathfrak{R}_1)/(\mu s)$ evaluates to u_c at x, so it leaves every fingerprint unchanged. Hence each label has one fingerprint before the reset and one after. Since identical symbolic functions have identical fingerprints, the two executions differ only if two labels that B keeps distinct receive the same fingerprint. For a pair of labels, insert the prescribed values into the difference of their exponents. This gives an affine function of the free fingerprints which, if not identically zero, vanishes with probability at most $1/q$ (the degree-one case of Schwartz–Zippel). It is indeed nonzero in both periods:

1. *Before the reset.* If u_{c^*} has a nonzero coefficient, the difference is nonzero since u_{c^*} is still free. Otherwise the reset does not affect it, so a zero difference would be a symbolic collision after all substitutions in $\mathcal{T}$, and B would have aborted.

2. *At or after the reset.* Additionally replace u_{c^*} by $u^+_{c^*}$. This matches the substitution of B for the programmed signature, so again a zero difference would make B abort. If there is no reset, this event is empty.

Each period involves at most $\binom{n_\pi}{2}$ pairs, including pairs formed when a new label is inserted; labels of consumed presignatures remain in π. This also covers the zero cases in Equation (3): w is the fingerprint of $\hat{R}$ and $A = sw$, so either zero is a collision with the identity. For the programmed signature, such a collision after the reset is detected by the rebuilt dictionary.

Conclusion. If $\varepsilon_{\mathrm{fp}} < 1$, the union bound gives

$$\frac{\Pr[\mathsf{B}_{\mathrm{lin}} \text{ produces } \mathcal{T}]}{\Pr[\mathsf{B} \text{ produces } \mathcal{T}]} \ge \left(1 - \frac{t}{q}\right)\left(1 - \frac{2\binom{n_\pi}{2}}{q}\right) \ge 1 - \varepsilon_{\mathrm{fp}}.$$

For $\varepsilon_{\mathrm{fp}} = 1$, the claimed bound is trivial. Summing over successful transcripts and averaging over the fixed choices proves the success bound.

Running Time. The stored definitions, the backward pass, and the single rebuild each cost $O(t)$, and each ordinary signature requires constant bookkeeping. Modifier records are stored by presignature, and each is inserted and removed once. This proves the time and storage bounds. □